NAME:G CHANIKYA

USN:23BTRCA055

HCL ID:HCLTFP2263489

ASSESSMENT-2

Model Question Paper

Cloud Data Engineer

Jain University

TOTAL: 100 MARKS

Time Limit: 3 Hours

SECTION A: ADVANCED SQL || Maximum Marks: 20

Question 1: Database Design and Advanced Queries (8 Marks)

Create a database named EnterpriseDB.

Create the following tables.

Create the following tables.

Employees

Column	Description
EmployeeID	Unique employee identifier
EmployeeName	Employee name
DepartmentID	Department identifier
Salary	Monthly salary
JoiningDate	Date of joining
ManagerID	Employee ID of the manager

Departments

Column	Description
DepartmentID	Unique department identifier
DepartmentName	Department name
Location	Department location

Requirements:

• Define appropriate data types and constraints.

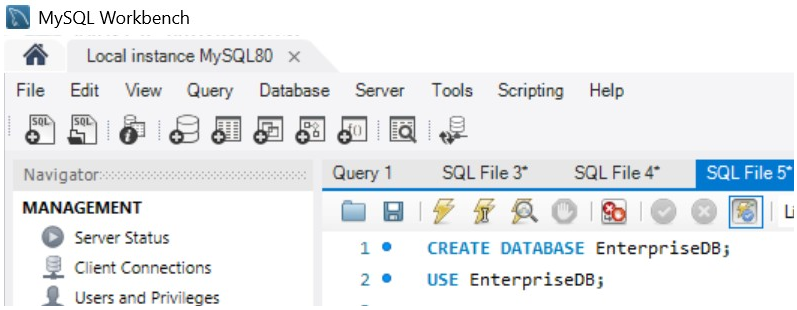
• Set primary keys and an appropriate foreign key.

• Insert at least 15 employee records and five department records.

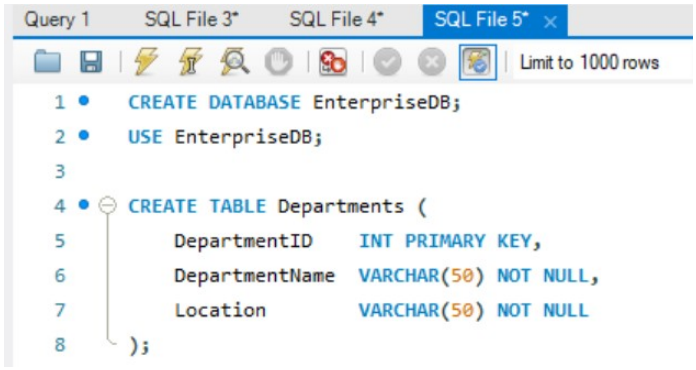
• Include employees with different salaries, departments, and managers.

• Include at least one department without employees.

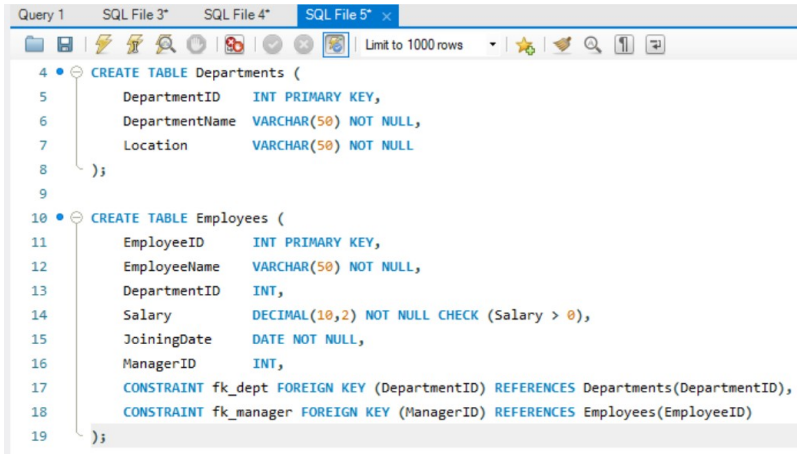
Create a database named EnterpriseDB:



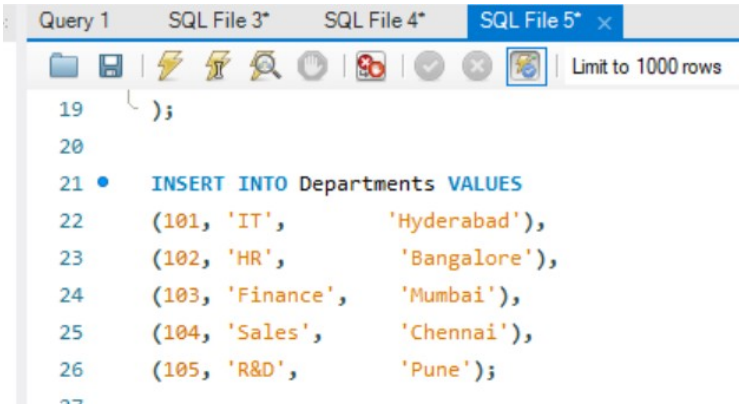
Departments Table:



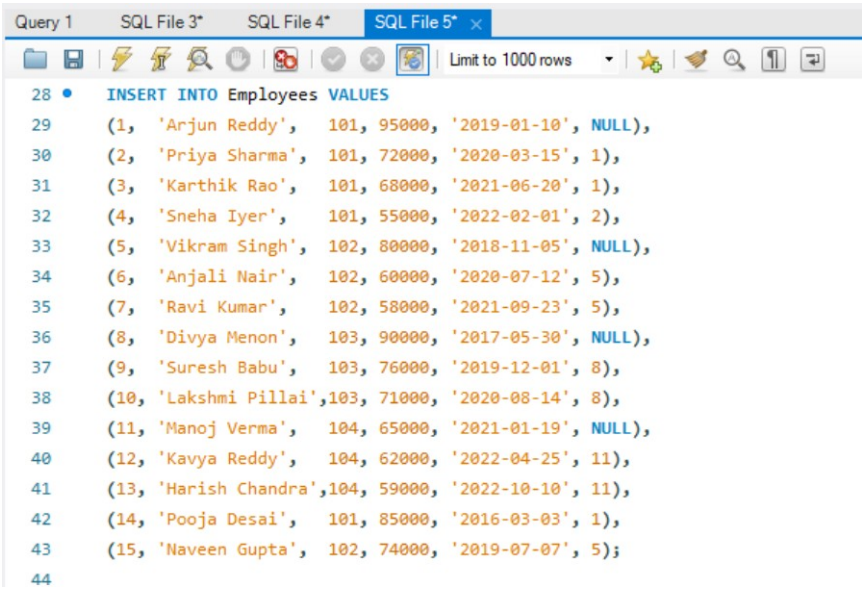
Employees Table:



INSERT INTO Departments:



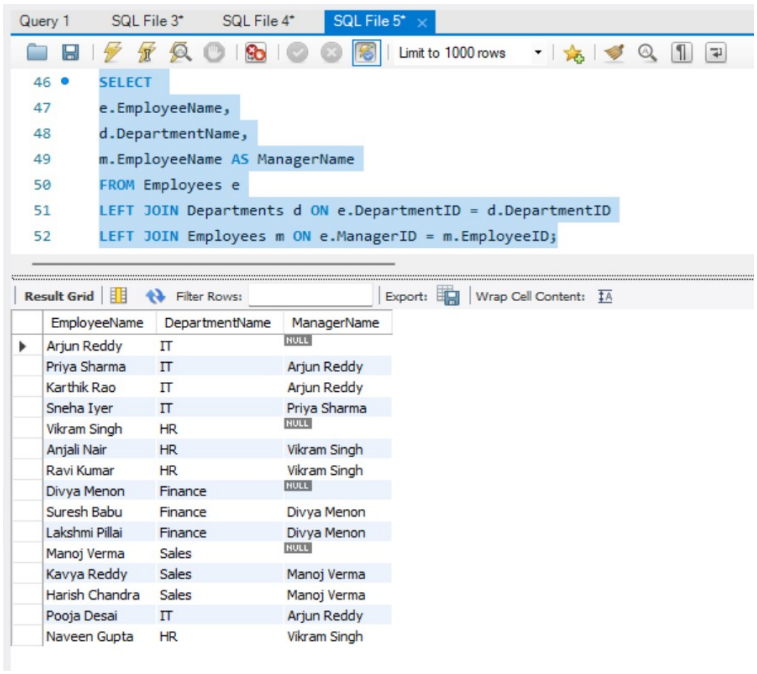
INSERT INTO Employees:



Tasks:

a. Display employee names, department names, and manager names using joins. (2 marks)

```
SELECT
e.EmployeeName,d.DepartmentName,m.EmployeeName AS ManagerName
FROM Employees e
LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

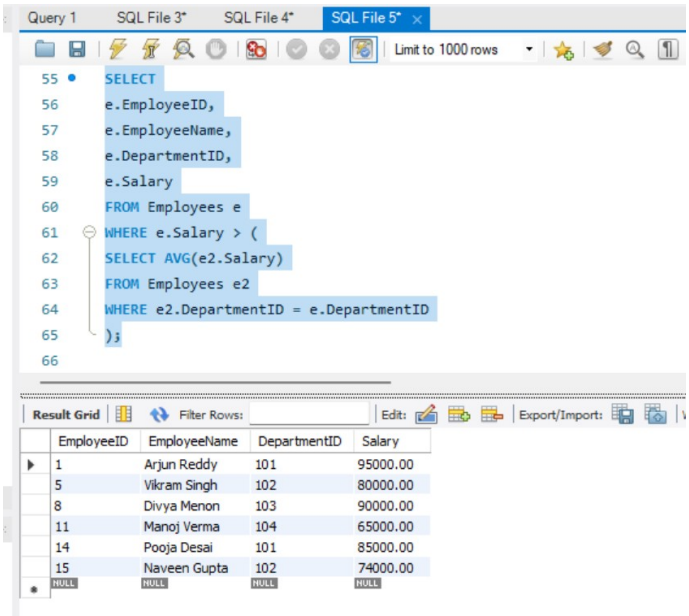


```
SELECT
e.EmployeeName,
d.DepartmentName,
m.EmployeeName AS ManagerName
FROM Employees e
LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

EmployeeName	DepartmentName	ManagerName
Arjun Reddy	IT	NULL
Priya Sharma	IT	Arjun Reddy
Karthik Rao	IT	Arjun Reddy
Sneha Iyer	IT	Priya Sharma
Vikram Singh	HR	NULL
Anjali Nair	HR	Vikram Singh
Ravi Kumar	HR	Vikram Singh
Divya Menon	Finance	NULL
Suresh Babu	Finance	Divya Menon
Lakshmi Pillai	Finance	Divya Menon
Manoj Verma	Sales	NULL
Kavya Reddy	Sales	Manoj Verma
Harish Chandra	Sales	Manoj Verma
Pooja Desai	IT	Arjun Reddy
Naveen Gupta	HR	Vikram Singh

b. Find employees earning more than their department's average salary. (2 marks)

```
SELECT
e.EmployeeID,e.EmployeeName,e.DepartmentID,e.Salary
FROM Employees e
WHERE e.Salary > (
SELECT AVG(e2.Salary)
FROM Employees e2
WHERE e2.DepartmentID = e.DepartmentID
);
```

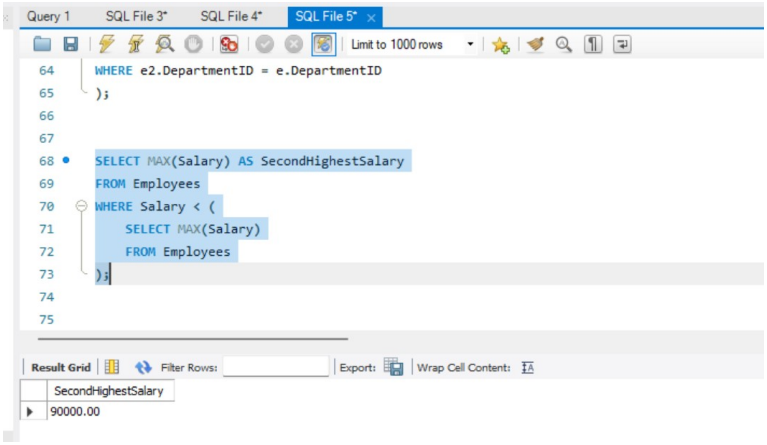


```
SELECT
e.EmployeeID,
e.EmployeeName,
e.DepartmentID,
e.Salary
FROM Employees e
WHERE e.Salary > (
SELECT AVG(e2.Salary)
FROM Employees e2
WHERE e2.DepartmentID = e.DepartmentID
);
```

EmployeeID	EmployeeName	DepartmentID	Salary
1	Arjun Reddy	101	95000.00
5	Vikram Singh	102	80000.00
8	Divya Menon	103	90000.00
11	Manoj Verma	104	65000.00
14	Pooja Desai	101	85000.00
15	Naveen Gupta	102	74000.00

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)

```
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employees
WHERE Salary < (
SELECT MAX(Salary)
FROM Employees
);
```



d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.

```
WITH RankedEmployees AS (
SELECT
e.EmployeeID,e.EmployeeName,d.DepartmentName,e.Salary,RANK() OVER (
PARTITION BY e.DepartmentID
ORDER BY e.Salary DESC
        ) AS SalaryRank
FROM Employees e
JOIN Departments d
ON e.DepartmentID = d.DepartmentID
)
SELECT
EmployeeID,EmployeeName,DepartmentName,Salary,SalaryRank
FROM RankedEmployees
WHERE SalaryRank <= 3
ORDER BY DepartmentName, SalaryRank, Salary DESC;
```

Query 1 | SQL File 3* | SQL File 4* | SQL File 5*

Limit to 1000 rows

```
71  SELECT
72  e.EmployeeID,e.EmployeeName,d.DepartmentName,e.Salary,RANK() OVER (
73  PARTITION BY e.DepartmentID
74  ORDER BY e.Salary DESC
75      ) AS SalaryRank
76  FROM Employees e
77  JOIN Departments d
78  ON e.DepartmentID = d.DepartmentID
79  )
80  SELECT
81  EmployeeID,EmployeeName,DepartmentName,Salary,SalaryRank
82  FROM RankedEmployees
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	EmployeeName	DepartmentName	Salary	SalaryRank
8	Divya Menon	Finance	90000.00	1
9	Suresh Babu	Finance	76000.00	2
10	Lakshmi Pillai	Finance	71000.00	3
5	Vikram Singh	HR	80000.00	1
15	Naveen Gupta	HR	74000.00	2
6	Anjali Nair	HR	60000.00	3
1	Arjun Reddy	IT	95000.00	1
14	Pooja Desai	IT	85000.00	2
2	Priya Sharma	IT	72000.00	3
11	Manoj Verma	Sales	65000.00	1
12	Kavya Reddy	Sales	62000.00	2

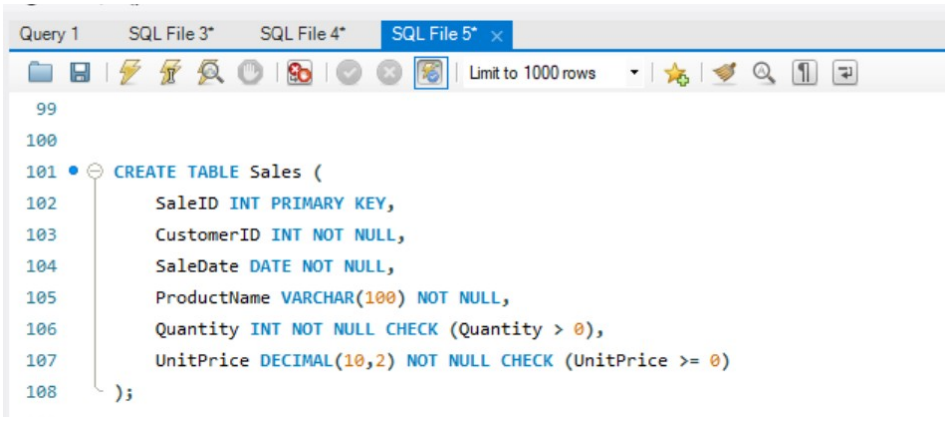
Question 2: Sales Analytics and Query Optimization (6 Marks)

Create a Sales table with the following columns:

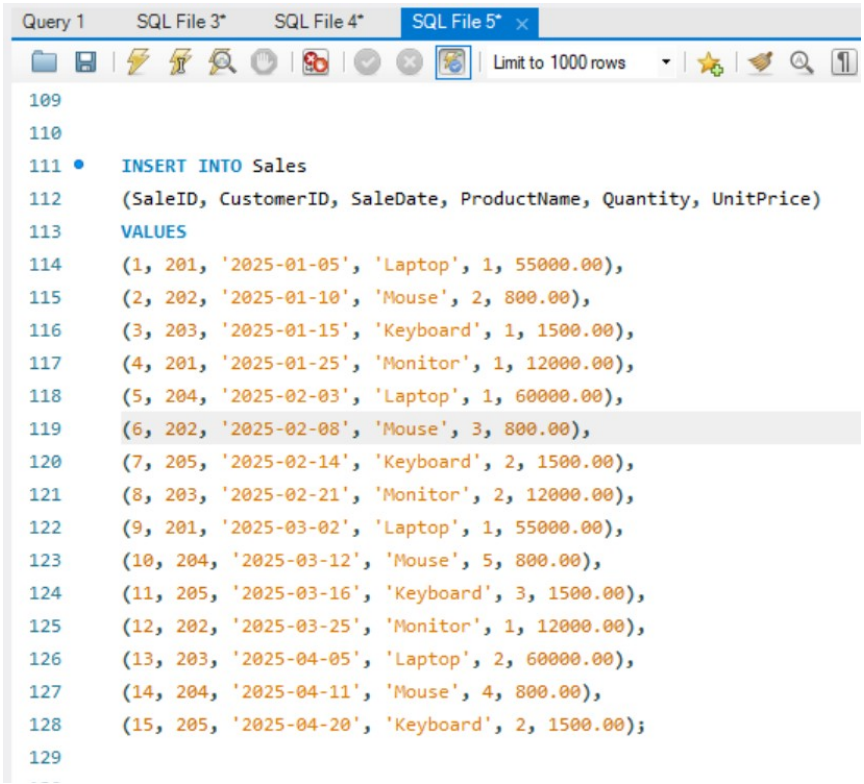
- SaleID
- CustomerID
- SaleDate
- ProductName
- Quantity
- UnitPrice

Insert at least 15 sample records covering multiple customers, products, and dates.

Sales table:



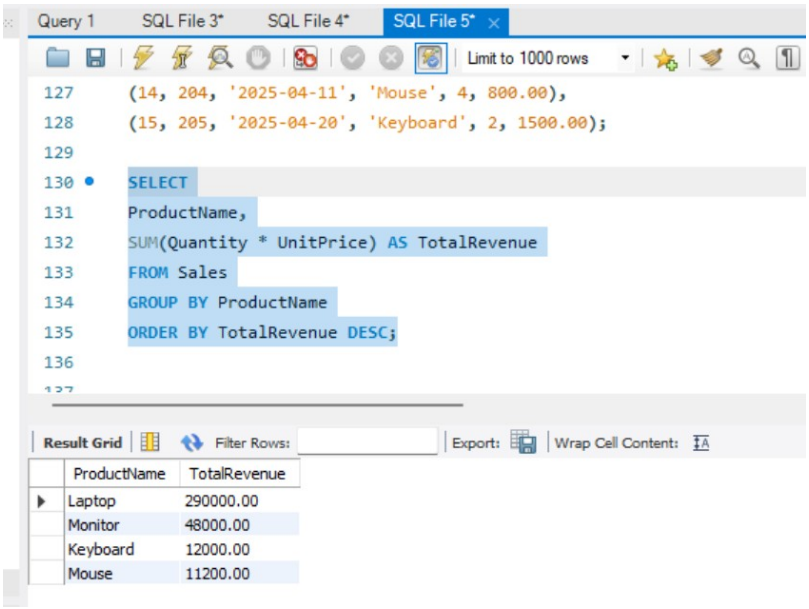
INSERT INTO Sales:



Write SQL queries to:

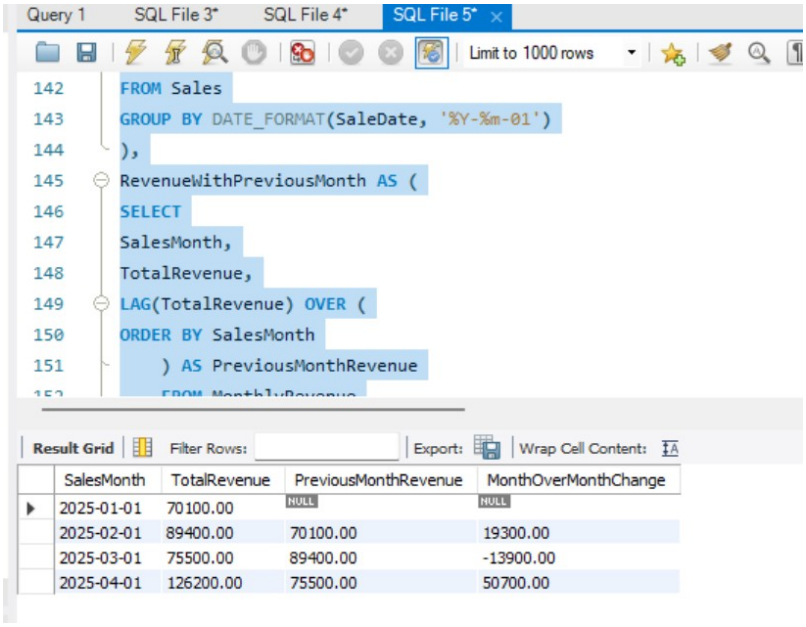
a. Calculate total revenue for each product. (2 marks)

```
SELECT
ProductName,
SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName
ORDER BY TotalRevenue DESC;
```



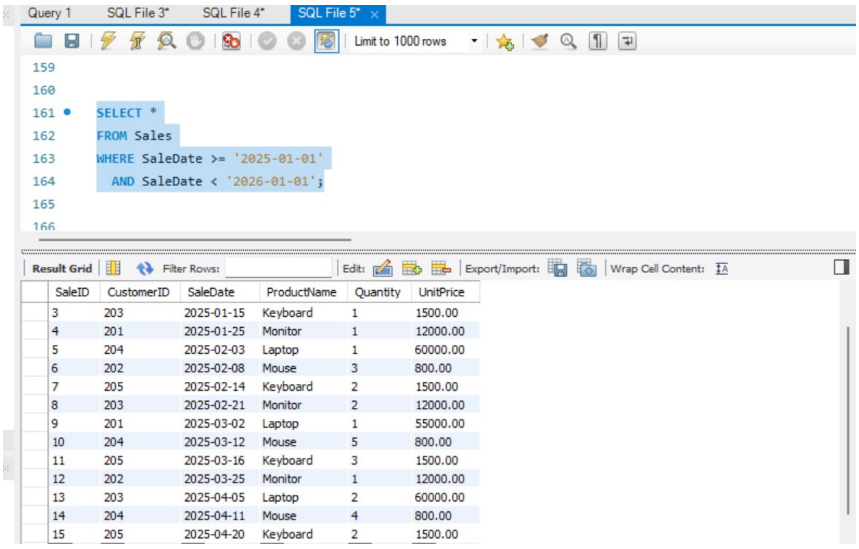
b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)

```
WITH MonthlyRevenue AS (
SELECT DATE_FORMAT(SaleDate, '%Y-%m-01') AS SalesMonth, SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY DATE_FORMAT(SaleDate, '%Y-%m-01')),
RevenueWithPreviousMonth AS (
SELECT SalesMonth, TotalRevenue, LAG(TotalRevenue) OVER (
ORDER BY SalesMonth
        ) AS PreviousMonthRevenue
   FROM MonthlyRevenue)
SELECT
   SalesMonth,TotalRevenue,PreviousMonthRevenue,TotalRevenue - PreviousMonthRevenue AS MonthOverMonthChange
FROM RevenueWithPreviousMonth
ORDER BY SalesMonth;
```



c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)
SELECT * FROM Employees
WHERE YEAR(JoiningDate) = 2025;
Assume JoiningDate may be indexed.

Optimised code:
SELECT *
FROM Sales
WHERE SaleDate >= '2025-01-01'
AND SaleDate < '2026-01-01';



Question 3: SQL Troubleshooting (6 Marks)

A company maintains millions of employee and transaction records. Its reporting queries are running slowly.

Perform the following tasks:

a. Explain how an execution plan helps identify expensive operations in a query. (2 marks)

An execution plan shows the steps the database optimizer will use to execute a query.

It helps identify expensive operations such as:

1. Full table scans on very large tables.
2. Large sorts (ORDER BY, GROUP BY).
3. Hash joins or nested loop joins with high row counts.
4. Missing indexes.
5. Expensive aggregate operations.
6. Poor join order.
7. Estimated rows much different from actual rows.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 marks)

```
CREATE INDEX idx_employee_department_salary

ON Employees(DepartmentID, Salary);

Code: SELECT *

FROM Employees

WHERE DepartmentID = 1

 AND Salary > 60000;
```

Reason:

1. DepartmentID is generally used as an equality filter: DepartmentID = 1.
2. Salary is generally used as a range condition: Salary > 60000.

The order (DepartmentID, Salary) is appropriate because the database can first locate employees within a department and then efficiently filter by salary.

This is better than using two separate indexes in many cases because a composite index can support both filtering conditions together.

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. (2 marks)

```
SELECT Email,COUNT(*) AS DuplicateCount
FROM Customers
WHERE Email IS NOT NULL
GROUP BY Email
HAVING COUNT(*) > 1;
```

This returns only email addresses that occur more than once.

SECTION B: LINUX || Maximum Marks: 15

Question 4: Log Analysis (5 Marks)

Assume /var/log/application.log contains millions of log entries in the following format: 2026-10-28 10:15:23 ERROR payment-service Transaction failed Write Linux commands to:

a.Find all lines containing ERROR. (1 mark)

```
grep "ERROR" /var/log/application.log
grep -i "ERROR" /var/log/application.log
```

b. Display the 20 most recent error entries in the file. (1 mark)

```
grep "ERROR" /var/log/application.log | tail -n 20
```

c. Count the total number of error entries. (1 mark)

```
grep "ERROR" /var/log/application.log | wc -l
```

d. Extract error entries for a specified date into a separate file. (1 mark)

```
grep "^2026-10-28.*ERROR" /var/log/application.log > errors_2026-10-28.log
```

e.Explain how you would calculate the number of errors occurring in each hour. (1 mark)

```
grep "ERROR" /var/log/application.log | \
awk '{print $1 " " substr($2, 1, 2) ":00"}' | \sort | uniq -c
```

Question 5: Permissions and System Troubleshooting (5 Marks)

A Linux server hosts an application that processes confidential financial reports. Perform the following tasks:

a. Create a group named finance and a report file, assuming you have the required privileges. (1 mark)

sudo groupadd finance

sudo touch /opt/reports/financial_report.txt

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. (1 mark)

sudo chmod 640 /opt/reports/financial_report.txt

c. Identify the processes consuming the most CPU. (1 mark)

top

ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 11

d. Display memory usage and filesystem disk utilization. (1 mark)

Memory usage: free -h

Filesystem disk usage: df -h

To view the size of a specific application directory: du -sh /opt/application

e. Find files larger than 100 MB under a specified application directory. (1 mark)

find /opt/application -type f -size +100M

Question 6: Bash Scripting (5 Marks)

Write a Bash script named backup_check.sh that:

1. Accepts source and destination directories as command-line arguments.
2. Validates that both arguments are provided.
3. Checks whether the source directory exists.

4. Creates the destination directory if it does not exist.

5. Creates a timestamped compressed archive of the source directory.

6. Displays a success message if the backup succeeds.

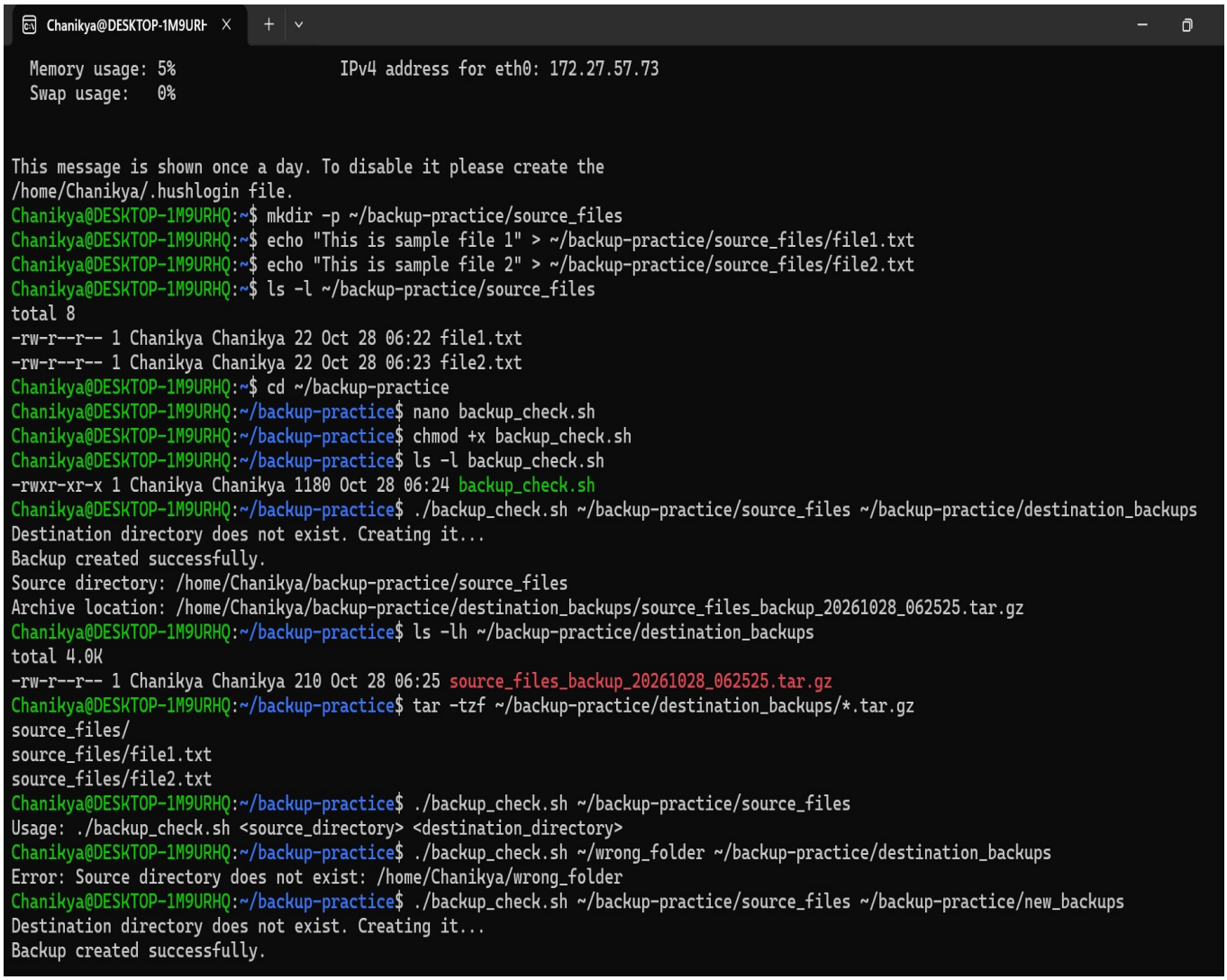
7. Displays an appropriate error message if the backup fails.

Expected Output: Submit the complete script and demonstrate its execution using valid and invalid arguments. Evaluation:

• Argument validation and directory checks: 2 marks

• Archive creation: 2 marks

• Error handling: 1 mark



SECTION C: PYSPARK DATA ENGINEERING

Maximum Marks: 20

Question 7: PySpark DataFrame Operations (6 Marks)

Create a PySpark DataFrame containing the following columns:

• EmployeeID

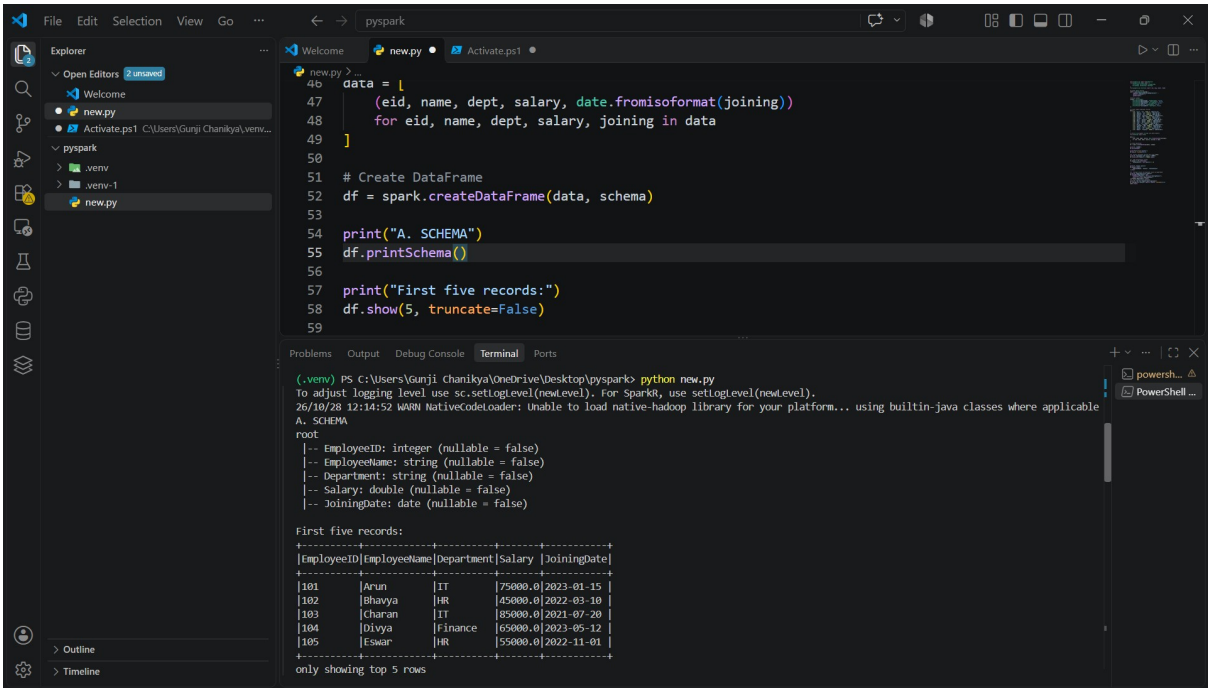
• EmployeeName • Department • Salary • JoiningDate

Insert at least 15 sample records. Write and execute a PySpark program to:

a. Display the schema and first five records. (1 mark)

```
from pyspark.sql import SparkSession
from pyspark.sql.types import (
    StructType, StructField, IntegerType,
    StringType, DoubleType, DateType
)
from pyspark.sql.functions import col, avg, count, round
# Create Spark session
spark = SparkSession.builder \
    .appName("EmployeeDataFrameOperations") \
    .master("local[*]") \
    .getOrCreate()
# Define schema
schema = StructType([
    StructField("EmployeeID", IntegerType(), False),
    StructField("EmployeeName", StringType(), False),
    StructField("Department", StringType(), False),
    StructField("Salary", DoubleType(), False),
    StructField("JoiningDate", DateType(), False)
])
# Create sample employee records
data = [
    (101, "Arun", "IT", 75000.0, "2023-01-15"),
```

```
    (102, "Bhavya", "HR", 45000.0, "2022-03-10"),
    (103, "Charan", "IT", 85000.0, "2021-07-20"),
    (104, "Divya", "Finance", 65000.0, "2023-05-12"),
    (105, "Eswar", "HR", 55000.0, "2022-11-01"),
    (106, "Farhan", "IT", 95000.0, "2020-02-18"),
    (107, "Gita", "Finance", 70000.0, "2021-09-25"),
    (108, "Hari", "Sales", 50000.0, "2023-08-14"),
    (109, "Indu", "Sales", 62000.0, "2022-06-30"),
    (110, "Jay", "IT", 60000.0, "2024-01-05"),
    (111, "Kiran", "Finance", 80000.0, "2020-12-11"),
    (112, "Lakshmi", "HR", 48000.0, "2023-04-22"),
    (113, "Manoj", "Sales", 72000.0, "2021-02-08"),
    (114, "Nisha", "IT", 68000.0, "2022-10-19"),
    (115, "Omkar", "Finance", 58000.0, "2024-02-01")
]
# Convert JoiningDate strings into date objects
from datetime import date
data = [
    (eid, name, dept, salary, date.fromisoformat(joining))
    for eid, name, dept, salary, joining in data
]
# Create DataFrame
df = spark.createDataFrame(data, schema)
print("A. SCHEMA")
df.printSchema()
print("First five records:")
df.show(5, truncate=False)
```



```
data = [
    (eid, name, dept, salary, date.fromisoformat(joining))
    for eid, name, dept, salary, joining in data
]

# Create DataFrame
df = spark.createDataFrame(data, schema)

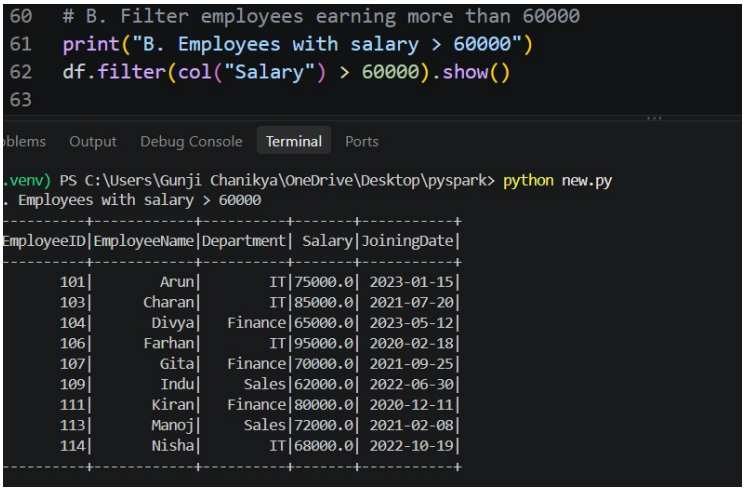
print("A. SCHEMA")
df.printSchema()

print("First five records:")
df.show(5, truncate=False)
```

```
(.venv) PS C:\Users\Gunji Chanikya\OneDrive\Desktop\pyspark> python new.py
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 12:14:52 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
A. SCHEMA
root
 |-- EmployeeID: integer (nullable = false)
 |-- EmployeeName: string (nullable = false)
 |-- Department: string (nullable = false)
 |-- Salary: double (nullable = false)
 |-- JoiningDate: date (nullable = false)

First five records:
+----------+------------+----------+-------+-----------+
|EmployeeID|EmployeeName|Department|Salary |JoiningDate|
+----------+------------+----------+-------+-----------+
|101       |Arun        |IT        |75000.0|2023-01-15 |
|102       |Bhavya      |HR        |45000.0|2022-03-10 |
|103       |Charan      |IT        |85000.0|2021-07-20 |
|104       |Divya       |Finance   |65000.0|2023-05-12 |
|105       |Eswar       |HR        |55000.0|2022-11-01 |
+----------+------------+----------+-------+-----------+
only showing top 5 rows
```

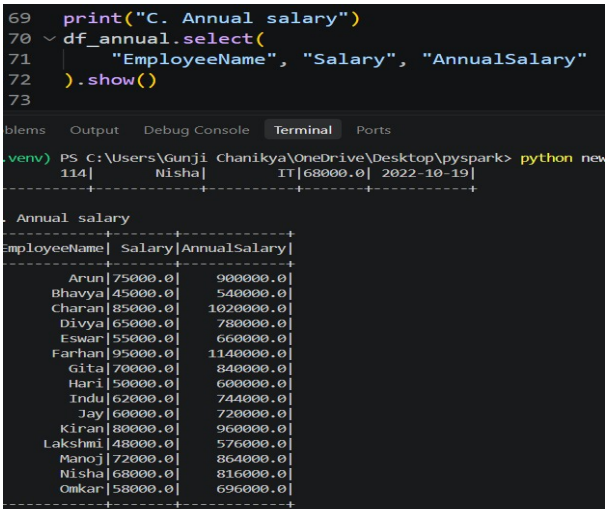
b. Filter employees whose monthly salary is greater than ₹60,000. (1 mark)



```
# B. Filter employees earning more than 60000
print("B. Employees with salary > 60000")
df.filter(col("Salary") > 60000).show()
```

```
.venv) PS C:\Users\Gunji Chanikya\OneDrive\Desktop\pyspark> python new.py
. Employees with salary > 60000
+----------+------------+----------+-------+-----------+
|EmployeeID|EmployeeName|Department| Salary|JoiningDate|
+----------+------------+----------+-------+-----------+
|       101|        Arun|        IT|75000.0| 2023-01-15|
|       103|      Charan|        IT|85000.0| 2021-07-20|
|       104|       Divya|   Finance|65000.0| 2023-05-12|
|       106|      Farhan|        IT|95000.0| 2020-02-18|
|       107|        Gita|   Finance|70000.0| 2021-09-25|
|       109|        Indu|     Sales|62000.0| 2022-06-30|
|       111|       Kiran|   Finance|80000.0| 2020-12-11|
|       113|       Manoj|     Sales|72000.0| 2021-02-08|
|       114|       Nisha|        IT|68000.0| 2022-10-19|
+----------+------------+----------+-------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary. (1 mark)



```
print("C. Annual salary")
df_annual.select(
    "EmployeeName", "Salary", "AnnualSalary"
).show()
```

```
.venv) PS C:\Users\Gunji Chanikya\OneDrive\Desktop\pyspark> python new
       114|       Nisha|        IT|68000.0| 2022-10-19|
+----------+------------+----------+-------+-----------+

. Annual salary
+------------+-------+------------+
|EmployeeName| Salary|AnnualSalary|
+------------+-------+------------+
|        Arun|75000.0|    900000.0|
|      Bhavya|45000.0|    540000.0|
|      Charan|85000.0|   1020000.0|
|       Divya|65000.0|    780000.0|
|       Eswar|55000.0|    660000.0|
|      Farhan|95000.0|   1140000.0|
|        Gita|70000.0|    840000.0|
|        Hari|50000.0|    600000.0|
|        Indu|62000.0|    744000.0|
|         Jay|60000.0|    720000.0|
|       Kiran|80000.0|    960000.0|
|     Lakshmi|48000.0|    576000.0|
|       Manoj|72000.0|    864000.0|
|       Nisha|68000.0|    816000.0|
|       Omkar|58000.0|    696000.0|
+------------+-------+------------+
```

d. Group employees by department and calculate the average salary and employee count. (2 marks)

```
74  # D. Average salary and employee count by department
75  print("D. Department-wise summary")
76  df.groupBy("Department").agg(
77      round(avg("Salary"), 2).alias("AverageSalary"),
78      count("*").alias("EmployeeCount")
79  ).orderBy("Department").show()
```

```
Problems  Output  Debug Console  Terminal  Ports

(.venv) PS C:\Users\Gunji Chanikya\OneDrive\Desktop\pyspark> python new.py
|      Omkar|58000.0|    696000.0|
+-----------+-------+------------+

D. Department-wise summary
+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|   Finance|      68250.0|            4|
|        HR|     49333.33|            3|
|        IT|      76600.0|            5|
|     Sales|     61333.33|            3|
+----------+-------------+-------------+
```

e. Display the five highest-paid employees. (1 mark)

```
80  # E. Five highest-paid employees
81  print("E. Top five highest-paid employees")
82  df.orderBy(col("Salary").desc()).show(5, truncate=False)
83  spark.stop()
84
```

```
Problems  Output  Debug Console  Terminal  Ports

(.venv) PS C:\Users\Gunji Chanikya\OneDrive\Desktop\pyspark> python new.py

E. Top five highest-paid employees
+----------+------------+----------+-------+-----------+
|EmployeeID|EmployeeName|Department|Salary |JoiningDate|
+----------+------------+----------+-------+-----------+
|106       |Farhan      |IT        |95000.0|2020-02-18 |
|103       |Charan      |IT        |85000.0|2021-07-20 |
|111       |Kiran       |Finance   |80000.0|2020-12-11 |
|101       |Arun        |IT        |75000.0|2023-01-15 |
|113       |Manoj       |Sales     |72000.0|2021-02-08 |
+----------+------------+----------+-------+-----------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation (7 Marks)

A company receives a CSV file named transactions.csv containing:

• TransactionID

• CustomerID

• TransactionDate

• Product

• Quantity

• UnitPrice

The file contains duplicate records, missing values, and incorrectly formatted dates. Write a PySpark program to:

```
from pyspark.sql import SparkSession
from pyspark.sql.types import (
   StructType, StructField, StringType, IntegerType,
   DoubleType
)
from pyspark.sql.functions import (
   col, trim, to_date, coalesce,
   sum as spark_sum
)
# Create Spark session
spark = SparkSession.builder \
   .appName("TransactionDataCleaning") \
   .master("local[*]") \
   .getOrCreate()
# Define CSV schema
schema = StructType([
   StructField("TransactionID", StringType(), True),
   StructField("CustomerID", StringType(), True),
   StructField("TransactionDate", StringType(), True),
   StructField("Product", StringType(), True),
   StructField("Quantity", IntegerType(), True),
```

```
    StructField("UnitPrice", DoubleType(), True)
])
```

a. Read the CSV file using an appropriate schema and header configuration. (1 mark)

```
df = spark.read \
    .option("header", True) \
    .option("mode", "PERMISSIVE") \
    .schema(schema) \
    .csv("transactions.csv")
print("A. Original data")
df.show(truncate=False)
print("Original record count:", df.count())
```

b. Identify and remove duplicate transactions using a suitable business key. (1 mark)

```
df = df.dropDuplicates(["TransactionID"])
print("After removing duplicate TransactionIDs:")
df.show(truncate=False)
```

c. Handle missing values in required fields and explain your chosen approach. (2 marks)

```
df = df.withColumn("TransactionID", trim(col("TransactionID"))) \
      .withColumn("CustomerID", trim(col("CustomerID"))) \
      .withColumn("Product", trim(col("Product")))

# Convert blank strings to null
for field in ["TransactionID", "CustomerID", "Product"]:
    df = df.withColumn(
        field,
        coalesce(
            # Preserve nonblank values; blanks become null
            # in the next expression below
            col(field),
```

```
            col(field)
        )
    )
from pyspark.sql.functions import when

for field in ["TransactionID", "CustomerID", "Product"]:
    df = df.withColumn(
        field,
        when(col(field) == "", None).otherwise(col(field))
    )

# Remove rows missing required business information
df = df.dropna(
    subset=[
        "TransactionID", "CustomerID", "Product",
        "TransactionDate", "Quantity", "UnitPrice"
    ]
)

# Reject nonpositive quantity or negative unit price
df = df.filter(
    (col("Quantity") > 0) & (col("UnitPrice") >= 0)
)
```

d. Convert TransactionDate into a date type and handle invalid dates. (1 mark)

```
df = df.withColumn(
    "TransactionDate",
    coalesce(
        to_date(col("TransactionDate"), "yyyy-MM-dd"),
        to_date(col("TransactionDate"), "dd-MM-yyyy"),
```

```
        to_date(col("TransactionDate"), "MM/dd/yyyy"),
        to_date(col("TransactionDate"), "dd/MM/yyyy")
    )
)
# Invalid dates become null; remove those records
df = df.dropna(subset=["TransactionDate"])
print("D. Cleaned transactions")
df.printSchema()
df.show(truncate=False)
print("Cleaned record count:", df.count())
```

e. Calculate revenue using Quantity × UnitPrice. (1 mark)

```
df = df.withColumn(
    "Revenue",
    col("Quantity") * col("UnitPrice")
)
print("E. Revenue for each transaction")
df.select(
    "TransactionID", "Product", "Quantity",
    "UnitPrice", "Revenue"
).show(truncate=False)
```

f. Calculate total revenue by product and display the results in descending order. (1 mark)

```
product_revenue = df.groupBy("Product").agg(
    spark_sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc())
print("F. Total revenue by product")
product_revenue.show(truncate=False)
spark.stop()
```

```
(.venv) PS C:\Users\Gunji Chanikya\OneDrive\Desktop\pyspark> python new.py
builtin-java classes where applicable
A. Original data
+-------------+----------+---------------+----------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product   |Quantity|UnitPrice|
+-------------+----------+---------------+----------+--------+---------+
|T101         |C01       |2026-01-10     |Laptop    |1       |50000.0  |
|T102         |C02       |11-01-2026     |Mobile    |2       |15000.0  |
|T103         |C03       |01/12/2026     |Headphones|3       |1000.0   |
|T104         |C04       |2026-01-13     |Mobile    |1       |15000.0  |
|T102         |C02       |11-01-2026     |Mobile    |2       |15000.0  |
|T105         |C05       |invalid-date   |Keyboard  |2       |500.0    |
|T106         |C06       |2026-01-15     |Laptop    |1       |50000.0  |
|T107         |NULL      |2026-01-16     |Mouse     |2       |400.0    |
|T108         |C08       |2026-01-17     |Keyboard  |3       |500.0    |
+-------------+----------+---------------+----------+--------+---------+

Original record count: 9
```

```
After removing duplicate TransactionIDs:
+-------------+----------+---------------+----------+--------+---------
|TransactionID|CustomerID|TransactionDate|Product   |Quantity|UnitPrice
+-------------+----------+---------------+----------+--------+---------
|T101         |C01       |2026-01-10     |Laptop    |1       |50000.0
|T102         |C02       |11-01-2026     |Mobile    |2       |15000.0
|T103         |C03       |01/12/2026     |Headphones|3       |1000.0
|T104         |C04       |2026-01-13     |Mobile    |1       |15000.0
|T105         |C05       |invalid-date   |Keyboard  |2       |500.0
|T106         |C06       |2026-01-15     |Laptop    |1       |50000.0
|T107         |NULL      |2026-01-16     |Mouse     |2       |400.0
|T108         |C08       |2026-01-17     |Keyboard  |3       |500.0
+-------------+----------+---------------+----------+--------+---------
```

```
D. Cleaned transactions
root
 |-- TransactionID: string (nullable = true)
 |-- CustomerID: string (nullable = true)
 |-- TransactionDate: date (nullable = true)
 |-- Product: string (nullable = true)
 |-- Quantity: integer (nullable = true)
 |-- UnitPrice: double (nullable = true)
```

Question 9: PySpark Joins and Performance (7 Marks)

A company maintains two datasets.

Customers

• CustomerID • CustomerName • City

Orders

• OrderID • CustomerID • OrderAmount • OrderDate

Write a PySpark program to:

```
from pyspark.sql import SparkSession
from pyspark.sql.types import (
   StructType, StructField, StringType,
   DoubleType, DateType, IntegerType
)
from pyspark.sql.functions import (
   col, sum as spark_sum, broadcast
)
from datetime import date
# Create Spark session
spark = SparkSession.builder \
   .appName("PySparkJoinsPerformance") \
   .master("local[*]") \
   .getOrCreate()
# Define Customers schema
customer_schema = StructType([
   StructField("CustomerID", IntegerType(), False),
   StructField("CustomerName", StringType(), False),
   StructField("City", StringType(), False)
])
```

```
# Sample customer records
customer_data = [
    (1, "Arun", "Bengaluru"),
    (2, "Bhavya", "Chennai"),
    (3, "Charan", "Hyderabad"),
    (4, "Divya", "Bengaluru"),
    (5, "Eswar", "Mumbai")
]
customers = spark.createDataFrame(
    customer_data, customer_schema
)
# Define Orders schema
order_schema = StructType([
    StructField("OrderID", IntegerType(), False),
    StructField("CustomerID", IntegerType(), False),
    StructField("OrderAmount", DoubleType(), False),
    StructField("OrderDate", DateType(), False)
])
# Sample order records
order_data = [
    (101, 1, 2500.0, date(2026, 1, 10)),
    (102, 2, 1800.0, date(2026, 1, 11)),
    (103, 1, 3200.0, date(2026, 1, 15)),
    (104, 4, 4500.0, date(2026, 1, 18)),
    (105, 3, 1500.0, date(2026, 1, 20))
]

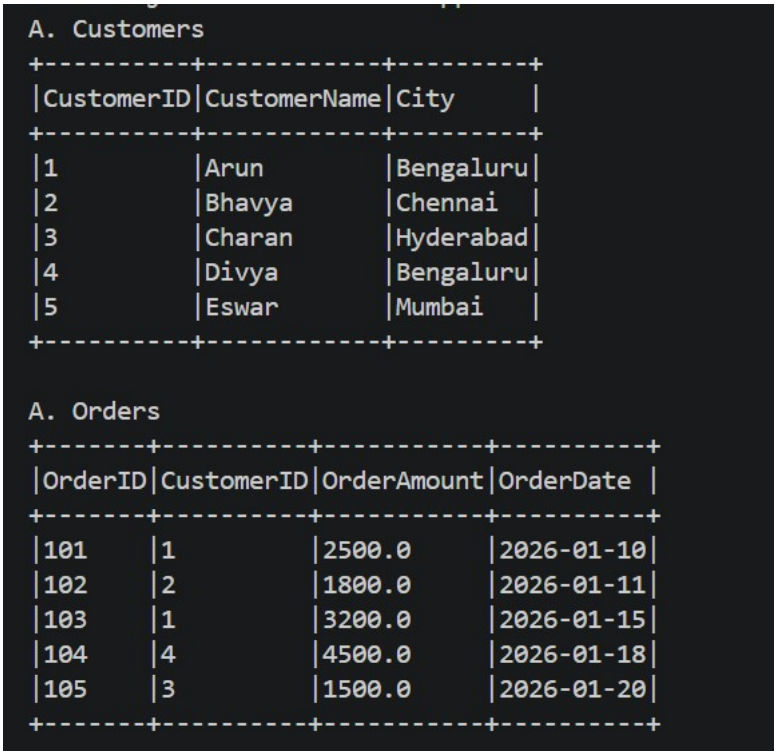
orders = spark.createDataFrame(
    order_data, order_schema
```

)

a. Load both datasets into DataFrames. (1 mark)

```
# A. Display both DataFrames
print("A. Customers")
customers.show(truncate=False)

print("A. Orders")
orders.show(truncate=False)
```



```
A. Customers
+----------+------------+---------+
|CustomerID|CustomerName|City     |
+----------+------------+---------+
|1         |Arun        |Bengaluru|
|2         |Bhavya      |Chennai  |
|3         |Charan      |Hyderabad|
|4         |Divya       |Bengaluru|
|5         |Eswar       |Mumbai   |
+----------+------------+---------+

A. Orders
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount|OrderDate |
+-------+----------+-----------+----------+
|101    |1         |2500.0     |2026-01-10|
|102    |2         |1800.0     |2026-01-11|
|103    |1         |3200.0     |2026-01-15|
|104    |4         |4500.0     |2026-01-18|
|105    |3         |1500.0     |2026-01-20|
+-------+----------+-----------+----------+
```

b. Join the datasets to display orders with customer details. (2 marks)

```
print("B. Orders with customer details")
customer_orders = customers.join(
   orders,
   on="CustomerID",
   how="inner"
)
```

```
customer_orders.select(
   "OrderID", "CustomerID", "CustomerName",
   "City", "OrderAmount", "OrderDate"
).show(truncate=False)
```

```
B. Orders with customer details
+-------+----------+------------+---------+-----------+----------+
|OrderID|CustomerID|CustomerName|City     |OrderAmount|OrderDate |
+-------+----------+------------+---------+-----------+----------+
|101    |1         |Arun        |Bengaluru|2500.0     |2026-01-10|
|103    |1         |Arun        |Bengaluru|3200.0     |2026-01-15|
|102    |2         |Bhavya      |Chennai  |1800.0     |2026-01-11|
|105    |3         |Charan      |Hyderabad|1500.0     |2026-01-20|
|104    |4         |Divya       |Bengaluru|4500.0     |2026-01-18|
+-------+----------+------------+---------+-----------+----------+
```

c. Identify customers who have never placed an order. (1 mark)

```
print("C. Customers without orders")
customers_without_orders = customers.join(
   orders,
   on="CustomerID",
   how="left_anti"
)
customers_without_orders.show(truncate=False)
```

```
C. Customers without orders
+----------+------------+------+
|CustomerID|CustomerName|City  |
+----------+------------+------+
|5         |Eswar       |Mumbai|
+----------+------------+------+
```

d. Calculate the total order value by city. (1 mark)

```
print("D. Total order value by city")
city_totals = customer_orders.groupBy("City").agg(
   spark_sum("OrderAmount").alias("TotalOrderValue")
```

).orderBy(col("TotalOrderValue").desc())

city_totals.show(truncate=False)

```
D. Total order value by city
+---------+---------------+
|City     |TotalOrderValue|
+---------+---------------+
|Bengaluru|10200.0        |
|Chennai  |1800.0         |
|Hyderabad|1500.0         |
+---------+---------------+
```

e. Explain when a broadcast join is useful and what conditions make it appropriate. (1 mark)

```
print("E. Broadcast join example")
broadcast_result = orders.join(
   broadcast(customers),
   on="CustomerID",
   how="inner"
)

broadcast_result.select(
   "OrderID", "CustomerID", "CustomerName",
   "City", "OrderAmount"
).show(truncate=False)
```

```
E. Broadcast join example
+-------+----------+------------+---------+-----------+
|OrderID|CustomerID|CustomerName|City     |OrderAmount|
+-------+----------+------------+---------+-----------+
|101    |1         |Arun        |Bengaluru|2500.0     |
|102    |2         |Bhavya      |Chennai  |1800.0     |
|103    |1         |Arun        |Bengaluru|3200.0     |
|104    |4         |Divya       |Bengaluru|4500.0     |
|105    |3         |Charan      |Hyderabad|1500.0     |
+-------+----------+------------+---------+-----------+
```

f. Explain the difference between transformations and actions, giving one example of each. (1 mark)

```
filtered_orders = orders.filter(
   col("OrderAmount") > 2000
)


# Action example: show()
print("F. Orders greater than 2000")
filtered_orders.show()
```

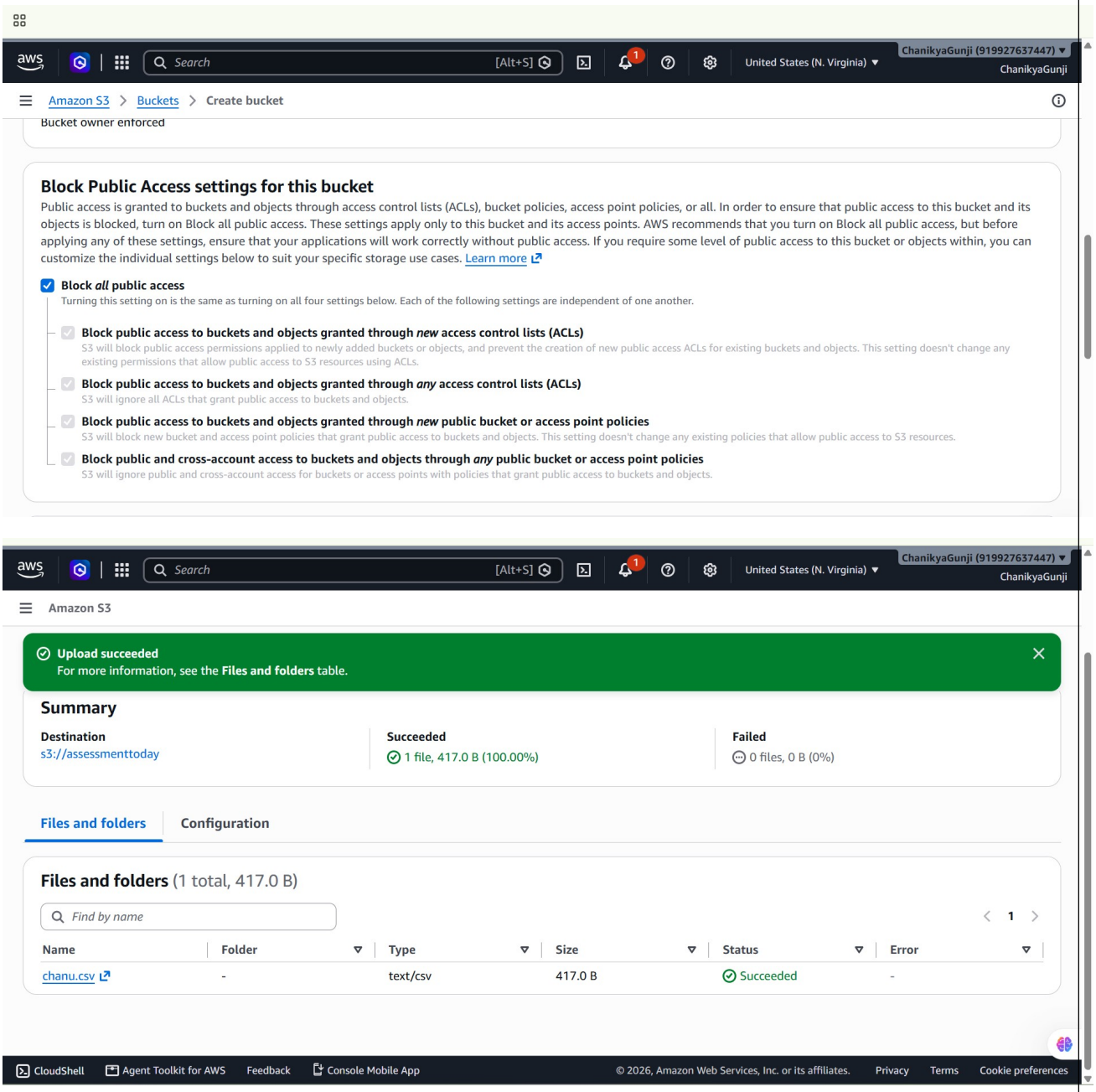
```
F. Orders greater than 2000
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|     2500.0|2026-01-10|
|    103|         1|     3200.0|2026-01-15|
|    104|         4|     4500.0|2026-01-18|
+-------+----------+-----------+----------+
```

SECTION D: AWS CLOUD SERVICES || Maximum Marks: 15

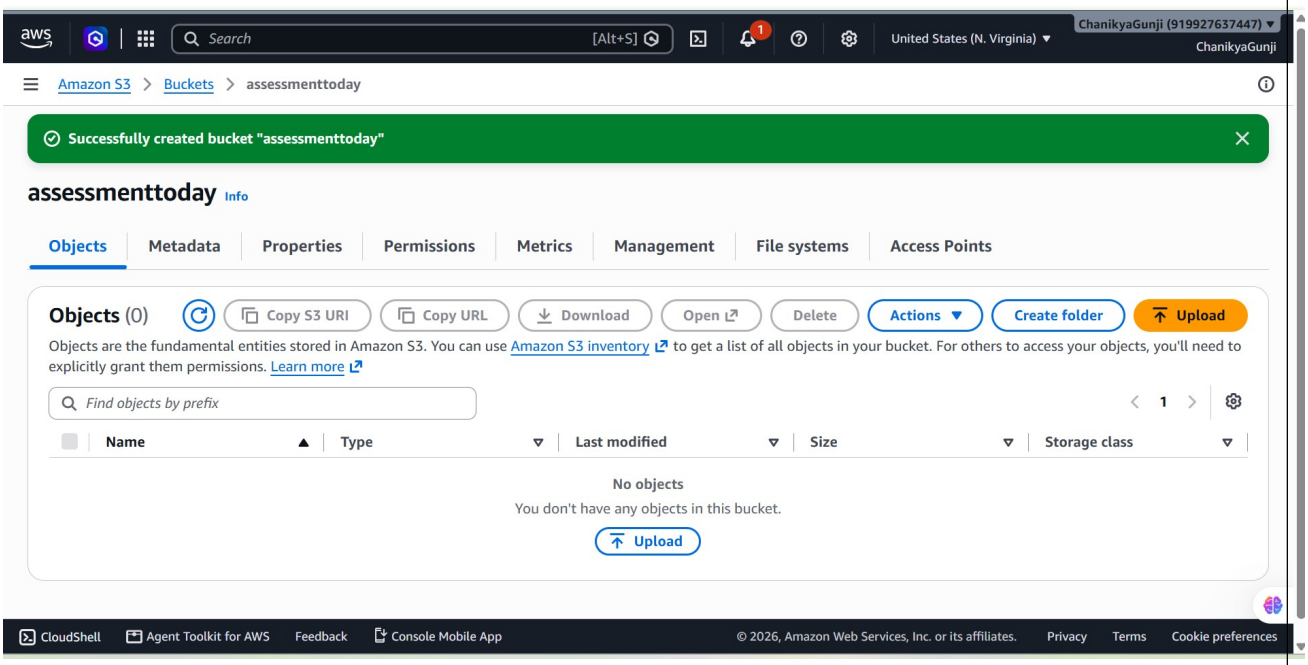
Question 10: S3 and IAM Security (5 Marks)

A company stores confidential employee reports in Amazon S3. Only authorized HR employees should be able to read these reports. Perform or document the following:

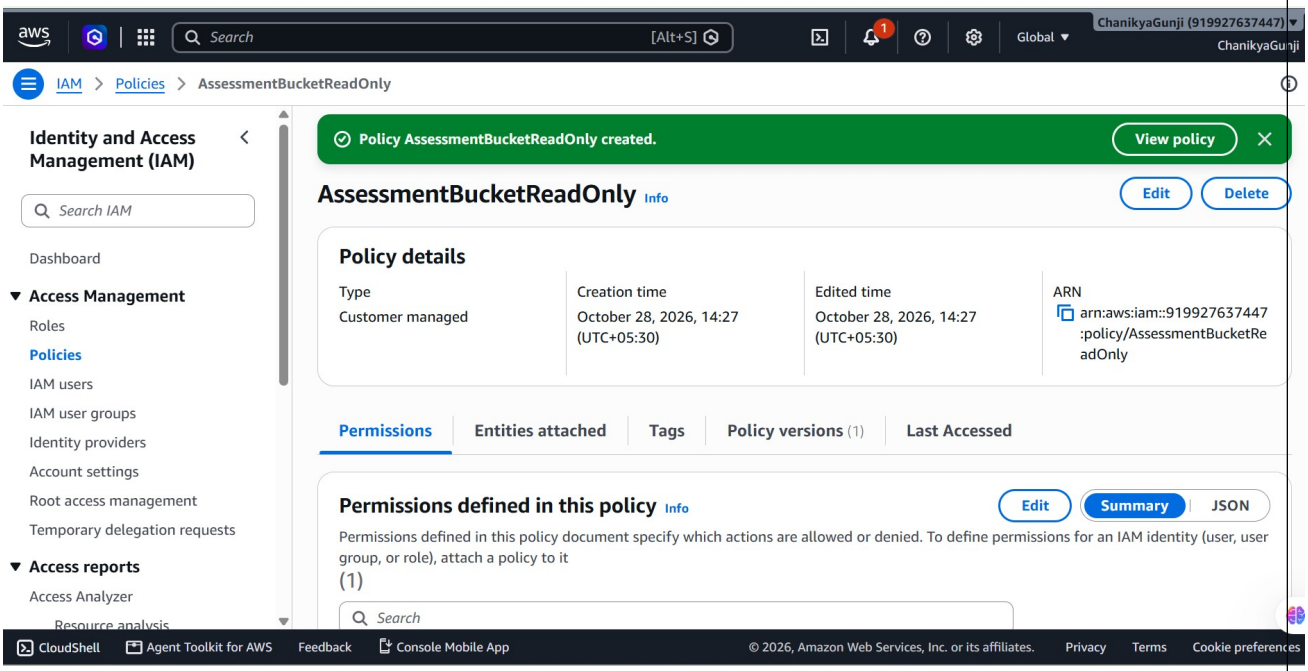
a. Create an S3 bucket with Block Public Access enabled and upload a sample file. (1 mark)


aws
Search
[Alt+S]
United States (N. Virginia)
ChanikyaGunji (919927637447)
ChanikyaGunji
Amazon S3 > Buckets > Create bucket
Bucket owner enforced
Block Public Access settings for this bucket
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. Learn more
Block all public access
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.
Block public access to buckets and objects granted through new access control lists (ACLs)
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
Block public access to buckets and objects granted through any access control lists (ACLs)
S3 will ignore all ACLs that grant public access to buckets and objects.
Block public access to buckets and objects granted through new public bucket or access point policies
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
Block public and cross-account access to buckets and objects through any public bucket or access point policies
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

aws
Search
[Alt+S]
United States (N. Virginia)
ChanikyaGunji (919927637447)
ChanikyaGunji
Amazon S3
Upload succeeded
For more information, see the Files and folders table.
Summary
Destination
s3://assessmenttoday
Succeeded
1 file, 417.0 B (100.00%)
Failed
0 files, 0 B (0%)
Files and folders
Configuration
Files and folders (1 total, 417.0 B)
Find by name
1
Name
Folder
Type
Size
Status
Error
chanu.csv
-
text/csv
417.0 B
Succeeded
-
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)



c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

Steps

1. Use an authorized HR identity to confirm that chanu.csv can be downloaded.
2. Sign in using a separate test identity that has no permission to read the bucket.
3. Try downloading the file using the AWS CLI:

aws s3 cp s3://assessmenttoday/chanu.csv .

4. Verify that the unauthorized identity receives an AccessDenied error.

   Expected result:

   An error occurred (AccessDenied)

   when calling the GetObject operation

5. Use a separate test identity with no other permissions granting access. If it can download the file, review its IAM policies, bucket policy, and other applicable permissions.

d. Explain how encryption at rest protects the stored data. (1 mark)

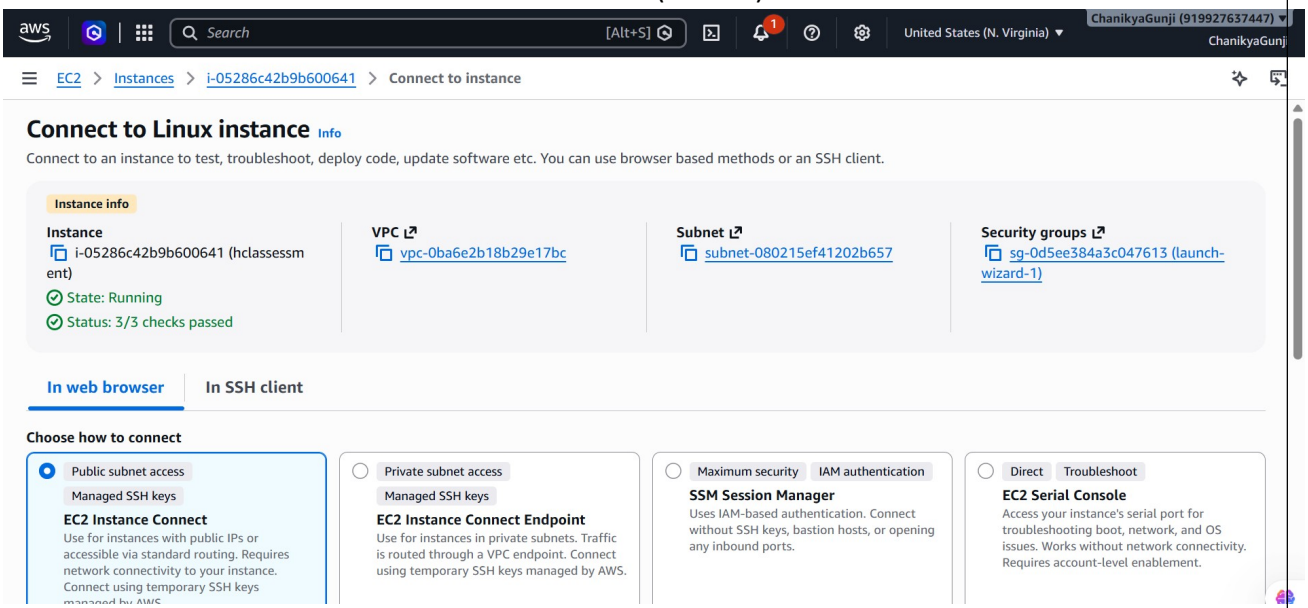
Amazon S3 supports server-side encryption, which encrypts stored objects so the underlying data is not stored as readable plaintext.

1. SSE-S3: Amazon S3 manages the encryption keys.
2. SSE-KMS: AWS Key Management Service manages the keys and provides additional access-control and auditing options.

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks)

A web application hosted on EC2 becomes slow, and users report connectivity problems. Perform or document the following:

a. Check the EC2 instance state and status checks. (1 mark)



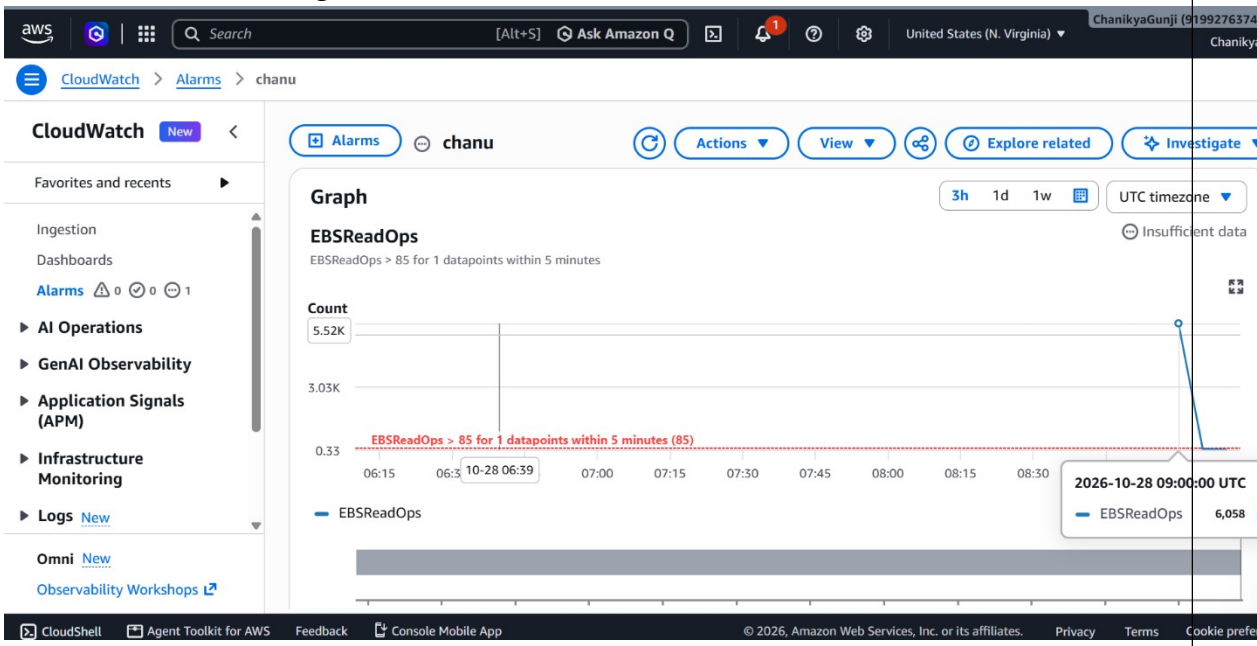
b. Explain how security groups and route tables can affect application connectivity. (1 mark)
Security groups act as virtual firewalls for EC2 instances.

- Allow inbound HTTP traffic on port 80 or HTTPS traffic on port 443, as required by the application.

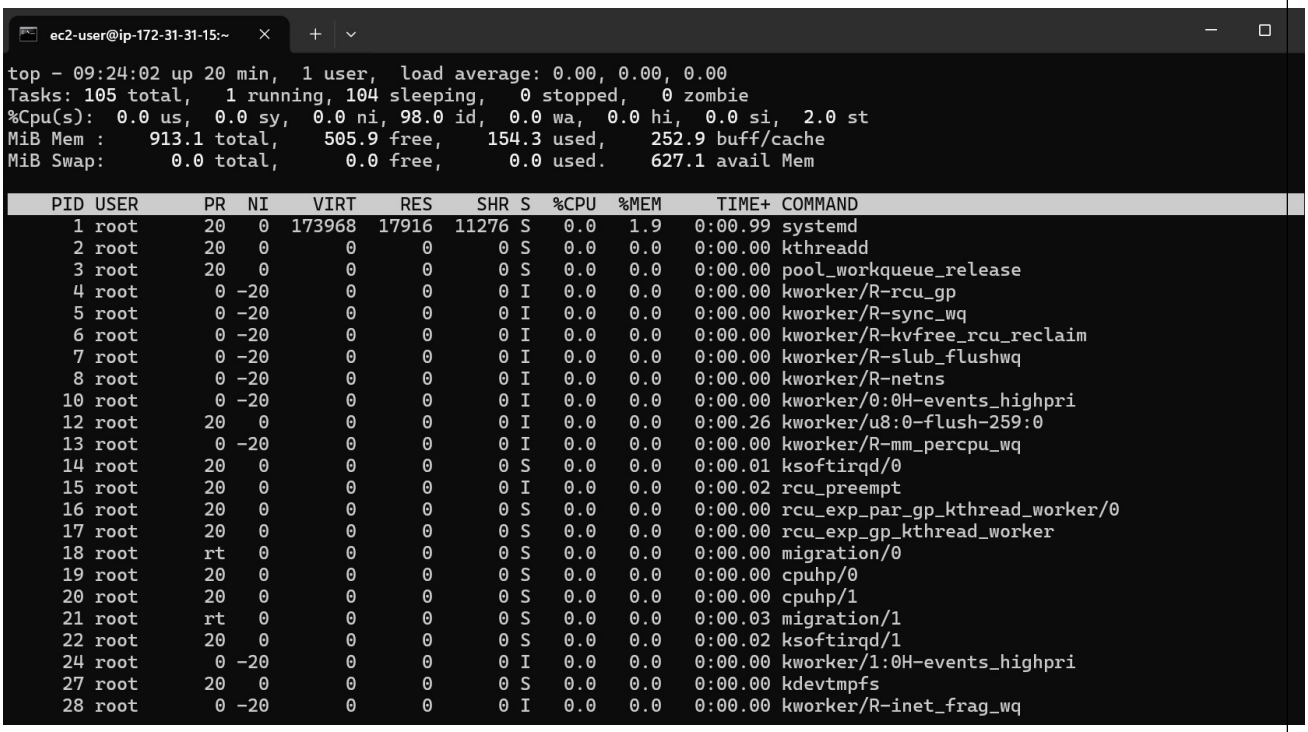
- Restrict SSH port 22 to trusted administrator IP addresses.
- Check outbound rules if the application needs to contact other services.
  Route tables determine how network traffic travels.
- A public subnet generally needs a route to an Internet Gateway for direct internet access.
- A private subnet may require a NAT Gateway for outbound internet access.
- Missing or incorrect routes can prevent users from reaching the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. (2 marks)

1. Open Amazon CloudWatch in the AWS Console.
2. Select Alarms → All alarms → Create alarm.
3. Click Select metric.
4. Choose EC2 → Per-Instance Metrics.
5. Select the required instance's CPUUtilization metric.
6. Set the statistic to Average and the period to 5 minutes.
7. Set the threshold type to Static and the condition to Greater than 85.
8. Configure the evaluation period as 2 out of 2 datapoints.
9. Optionally select an Amazon SNS topic to receive email alerts.
10. Name the alarm EC2-High-CPU-Alarm and click Create alarm.



d. Identify one method for investigating the root cause of high CPU utilization. (1 mark)
Use the Linux top command to identify processes consuming excessive CPU.



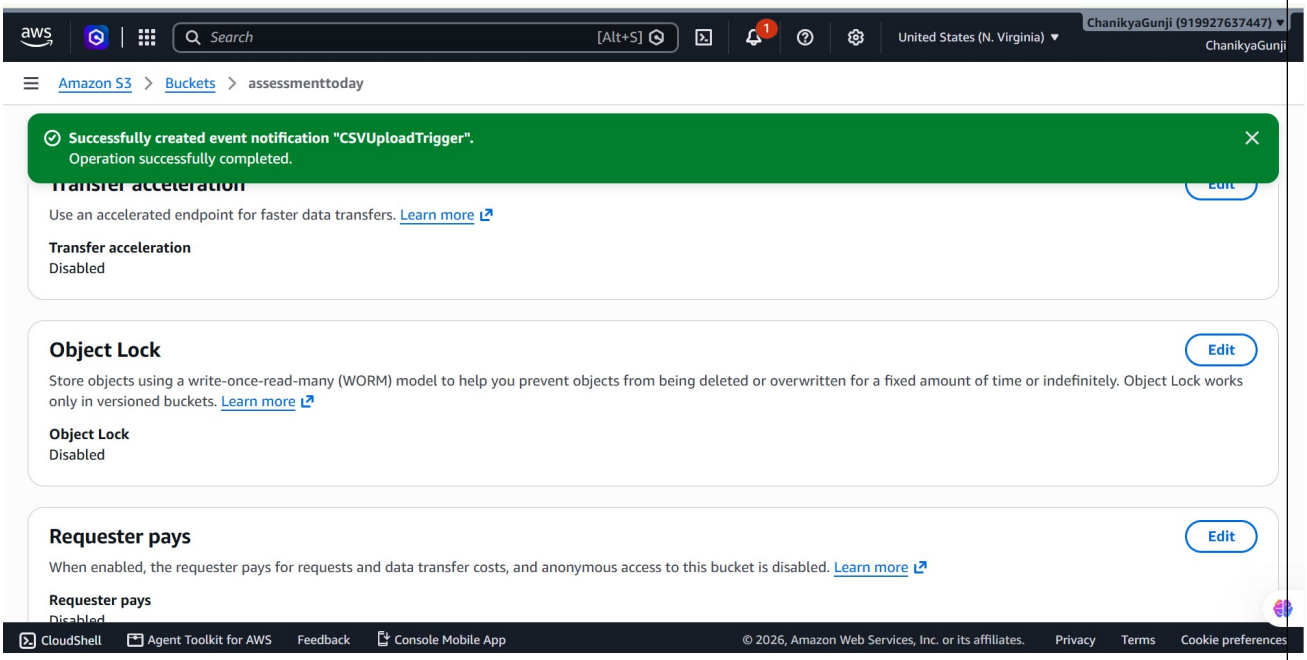
Question 12: Event-Driven Processing (5 Marks)

A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.

a. Identify a suitable service for executing the processing code. (1 mark)

AWS Lambda executes code automatically without requiring you to manage servers. It can validate and process CSV files uploaded to Amazon S3.

b. Explain how an S3 upload can trigger processing. (1 mark)

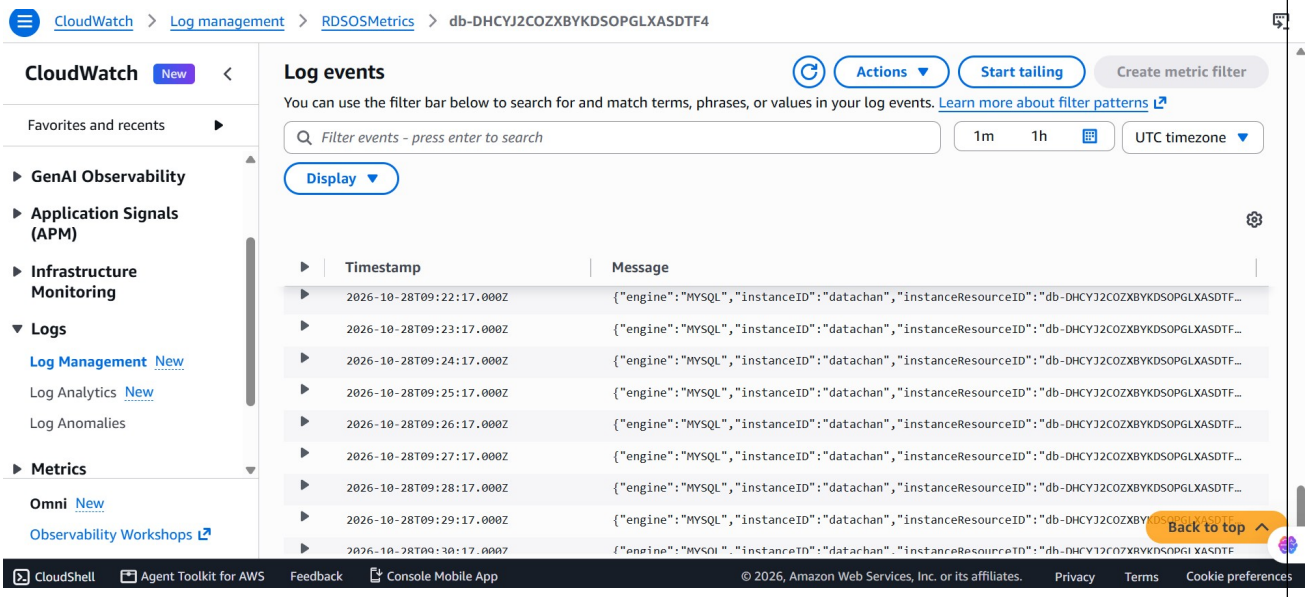


c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Action": "s3:GetObject",
   "Resource": "arn:aws:s3:::assessmenttoday/*"
  },
  {
   "Effect": "Allow",
   "Action": "s3:PutObject",
   "Resource": "arn:aws:s3:::your-output-bucket/*"
  },
  {
   "Effect": "Allow",
   "Action": [
    "logs:CreateLogStream",
    "logs:PutLogEvents"
```

```
    ],
    "Resource": "*"
   }
 ]
}
```

d. Explain how errors and processing logs can be captured. (1 mark)



e. Explain how to prevent duplicate processing or recursive triggering. (1 mark)

Duplicate processing: Track processed object keys and versions in DynamoDB and make processing idempotent, because S3 events can be delivered more than once.

Recursive triggering: Use a separate output bucket, or restrict notifications to an input prefix such as incoming/ while saving results under processed/

SECTION E: AWS STEP FUNCTIONS || Maximum Marks: 15

Question 13: State Machine Design (7 Marks)

A company needs to process sales files through the following stages:

1. Validate the input file.

2. Transform the data.

3. Calculate total sales by product.

4. Store the results in Amazon S3.

5. Send a completion notification.

Design an AWS Step Functions state machine to coordinate the workflow.

a. Identify suitable states for validation, processing, storage, and notification. (2 marks)

Suitable states:

Task: Runs a Lambda function or AWS service operation.

Choice: Checks whether the file is valid.

Fail: Stops the workflow for invalid input or an unrecoverable error.

Succeed: Marks the workflow as successfully completed.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)

The validation task returns a field such as "isValid": true or "isValid": false. The Choice state checks this value.

If isValid is true, the workflow proceeds to data transformation.

If isValid is false, it routes to a Fail state and stops.

c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)

Retry: Automatically tries a failed task again. It is useful for temporary issues, such as throttling or a brief service interruption.

Catch: Redirects a failed task to an error-handling state. It is useful when retries fail or the error cannot be recovered automatically.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)

A Task state can invoke a Lambda function using its ARN. Step Functions can also integrate with supported services such as Amazon S3, SNS, and AWS Glue. For example, Lambda can validate a CSV file, transform its data, and calculate product totals before results are stored in S3 and a notification is sent.

e. Explain how execution history helps troubleshoot a failed workflow. (1 mark)

Execution history records state transitions, task inputs and outputs, errors, and retry attempts. It helps identify which step failed and why, making troubleshooting easier.

Question 14: Step Functions Implementation (8 Marks)

A data engineering team has developed three processing functions:

• ValidateFile • TransformData • LoadResults

The functions must execute in sequence. Requirements:

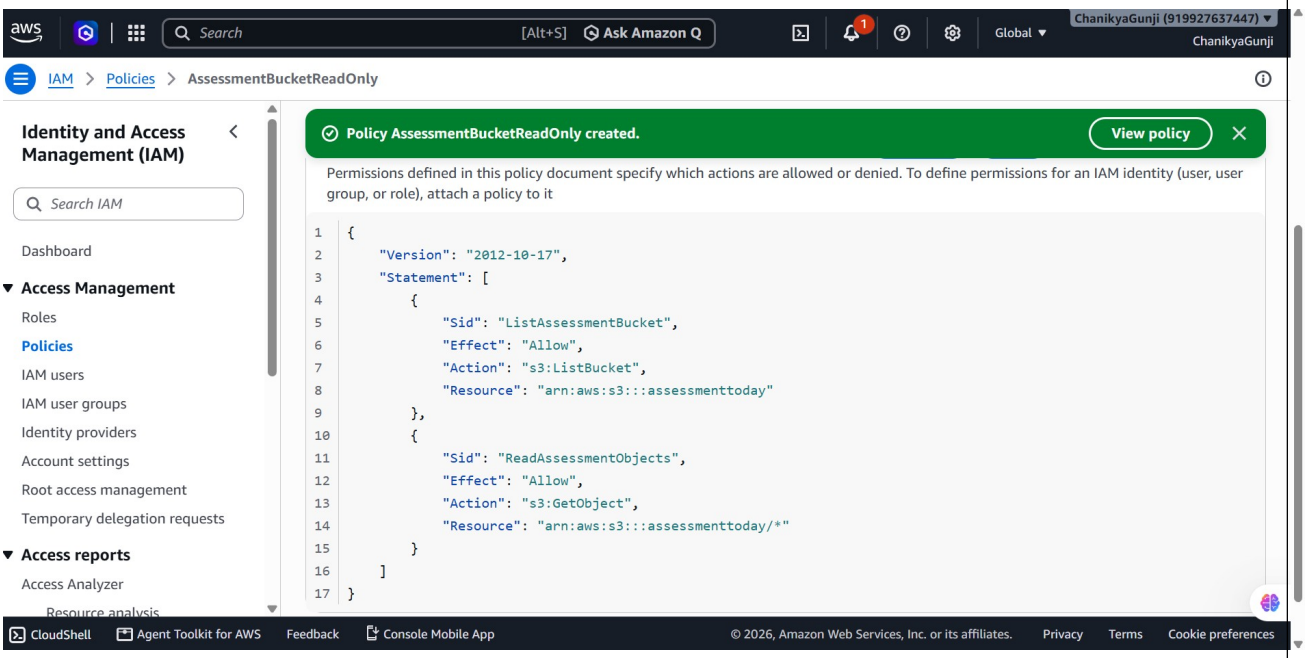
• If validation fails, the workflow must stop.

• If transformation fails temporarily, the workflow must retry.

• If loading fails permanently, the workflow must record the failure. Tasks:

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary. (3 marks)

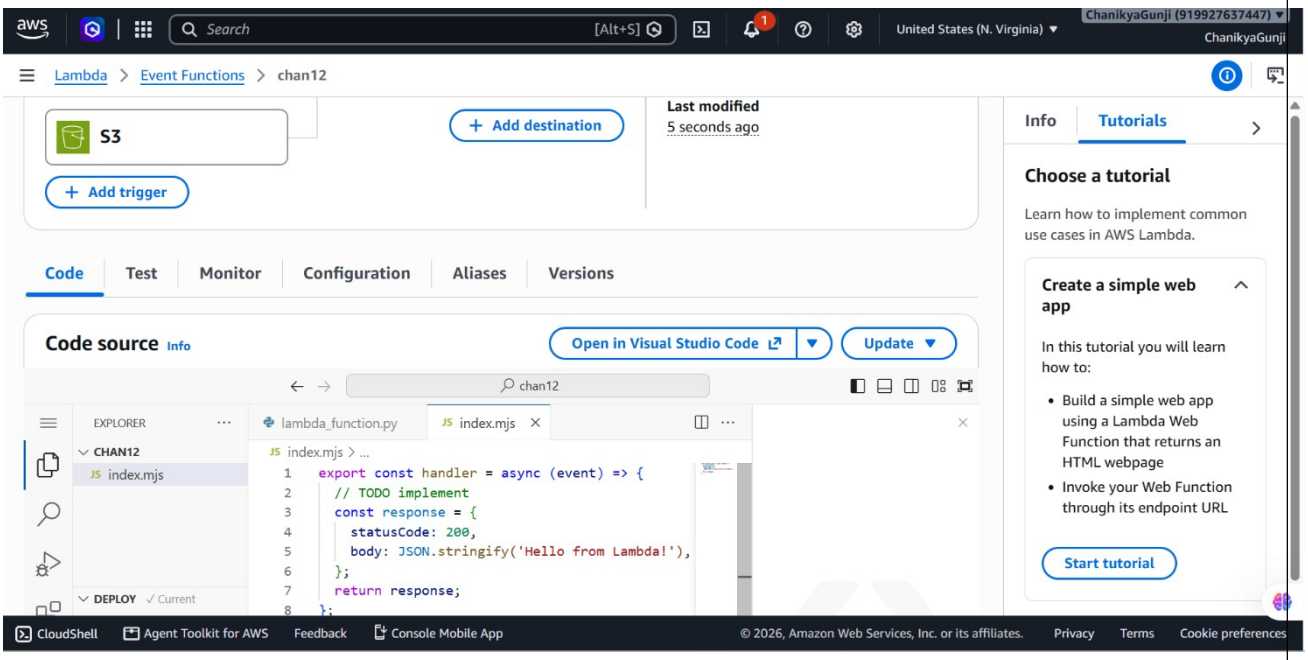
The workflow executes ValidateFile → TransformData → LoadResults.



Copy this JSON into the AWS Step Functions state machine editor. Replace the placeholder ARNs with your actual Lambda function ARNs.

b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts. (2 marks)

The retry delays are approximately 2, 4, and 8 seconds. Validation has no Retry or Catch rule, so a validation failure stops the workflow by default.



c. Configure a Catch transition for the loading task that routes to an error-handling state. (1 mark)

```
"Catch": [
 {
  "ErrorEquals": ["States.ALL"],
  "ResultPath": "$.loadError",
  "Next": "RecordLoadFailure"
 }
]
```

d. Explain how data from one task is passed to the next task. (1 mark)

By default, each Task receives the previous state's output as its input. The Task's result becomes the next state's input.

For example:

ValidateFile receives the input file location and returns validation details.

TransformData receives those details and returns transformed data or its S3 location.

LoadResults receives the transformed output and stores the results.

For a real workflow, keep large datasets in S3 and pass the S3 bucket/key or object location between tasks rather than passing an entire large CSV through the state machine.

e. Explain how you would test successful and failed executions. (1 mark)

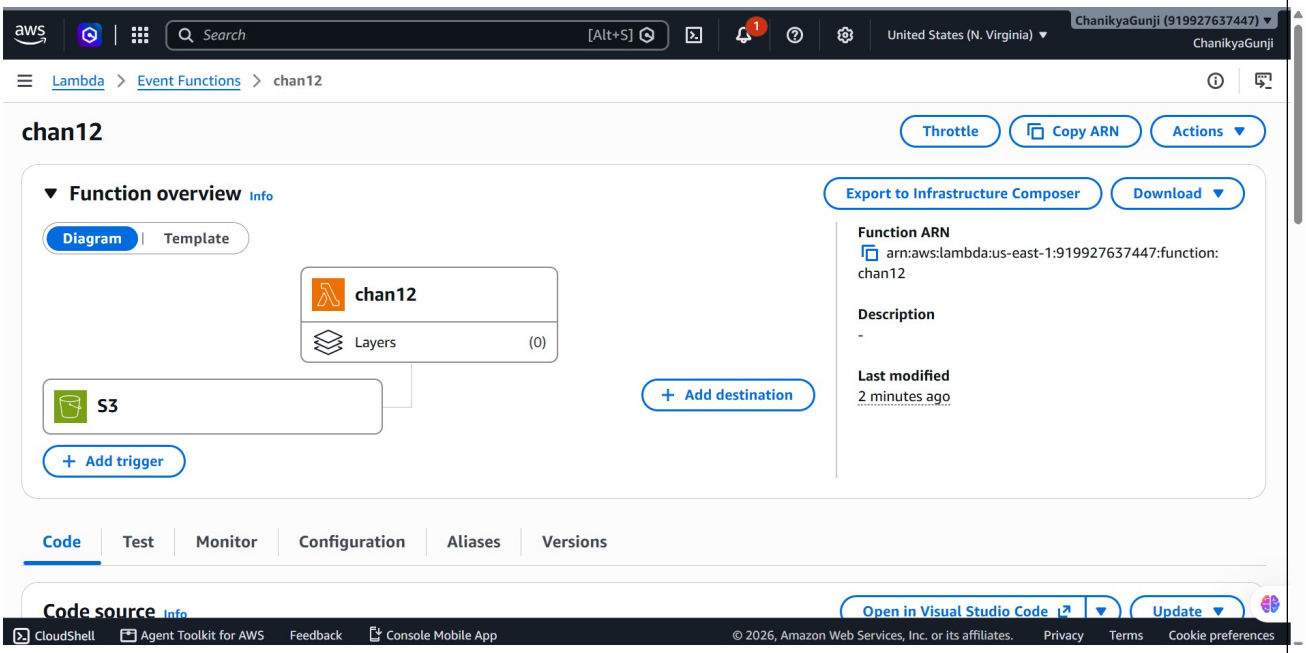
Test these scenarios:

Success: All three functions succeed; execution ends successfully.

Validation failure: ValidateFile raises an error; the workflow stops.

Temporary transformation failure: TransformData raises a retryable error; the workflow retries.

Loading failure: LoadResults fails; the error handler records the failure and the workflow ends in WorkflowFailed.

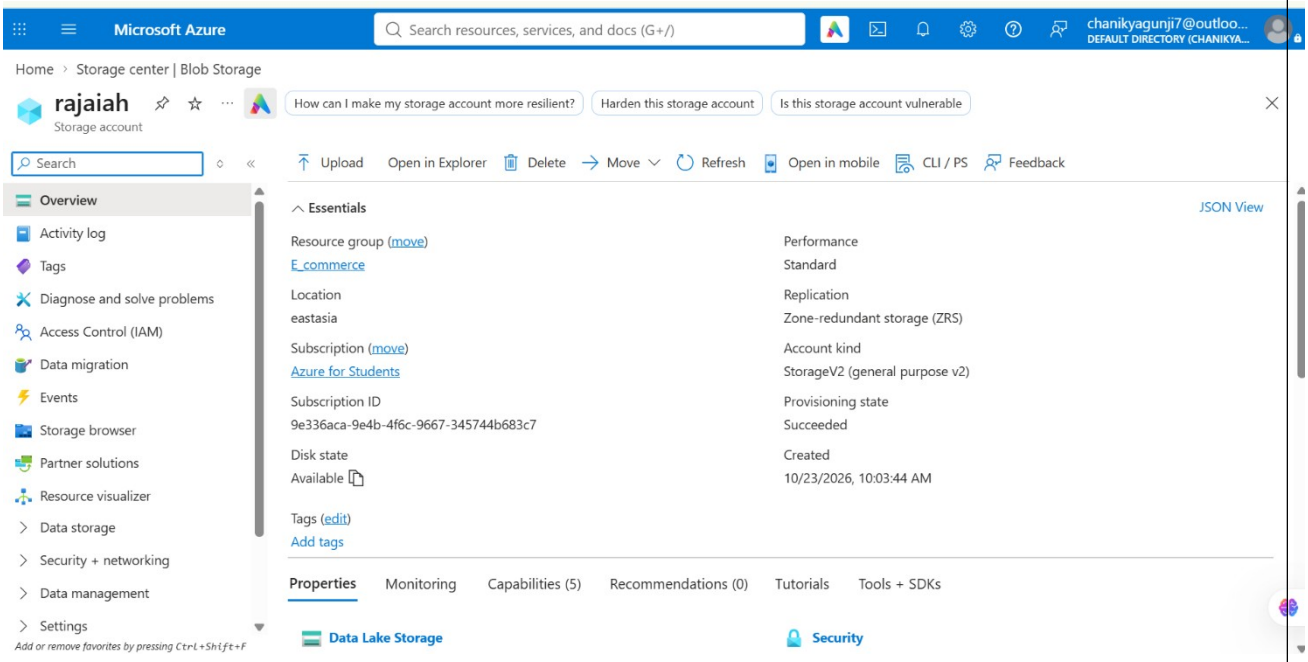


SECTION F: MICROSOFT AZURE || Maximum Marks: 15

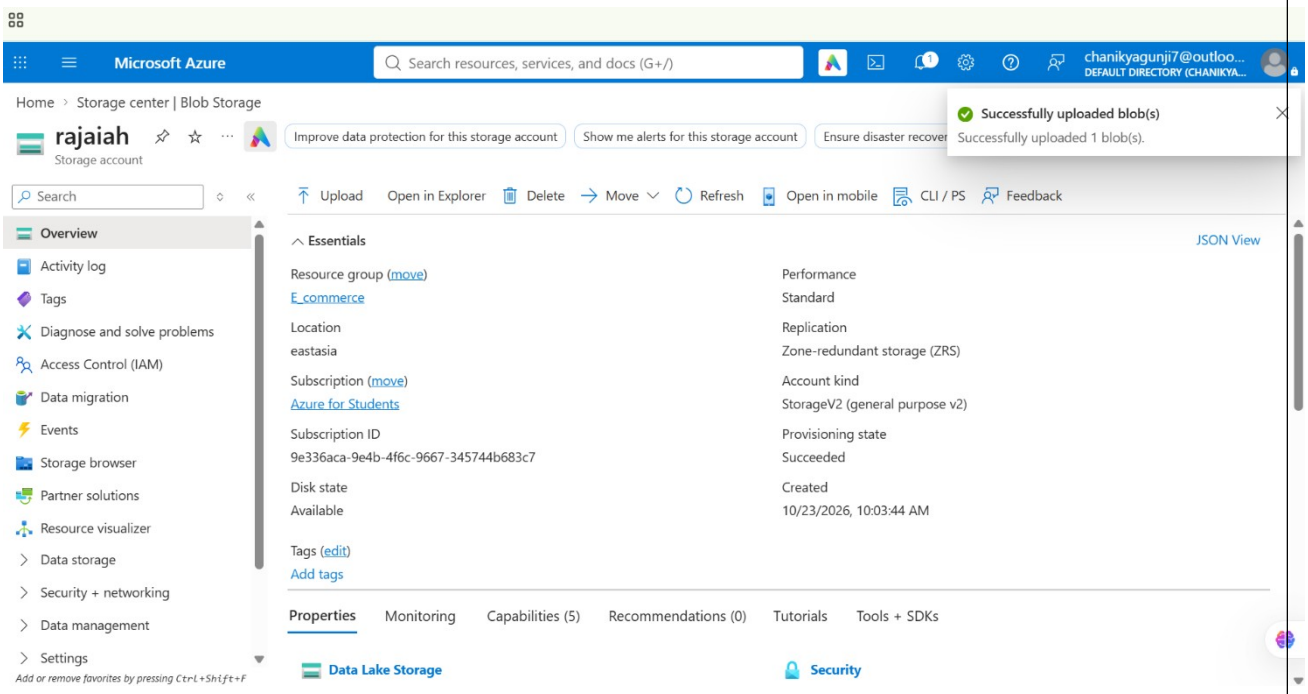
Question 15: Azure Data Lake Storage and Security (5 Marks)

A company needs to store confidential business data in Azure for analytics. Perform or document the following:

a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2. (1 mark)



b. Create a filesystem or container and upload a sample dataset. (1 mark)



c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data. (2 marks)

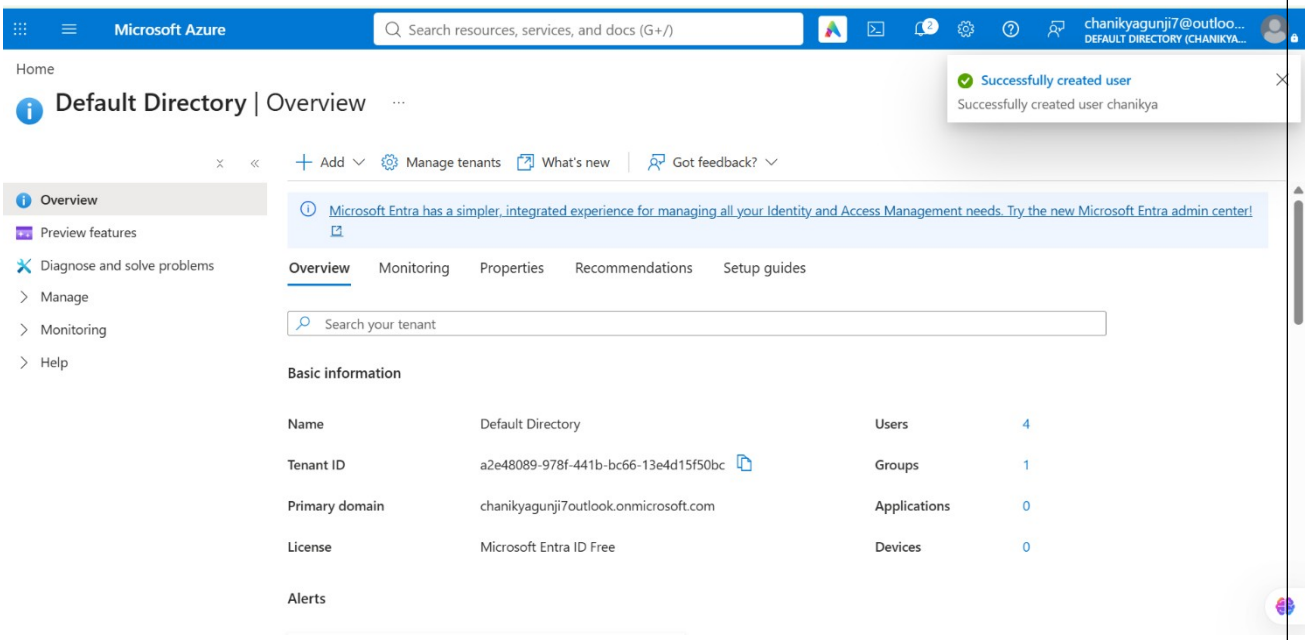
Microsoft Entra ID manages user identities, while Azure role-based access control (RBAC) determines what users can do with Azure resources.

For example:

Storage Blob Data Reader: Can read and download data.

Storage Blob Data Contributor: Can read, upload, and modify data.

Storage Blob Data Owner: Can access data and manage data permissions, subject to the applicable scope and authorization model.



d. Explain how you would verify that unauthorized users cannot access the dataset. (1 mark)

Give a test user only the permissions they require—or no data-access role.

Sign in as that user and attempt to access the dataset.

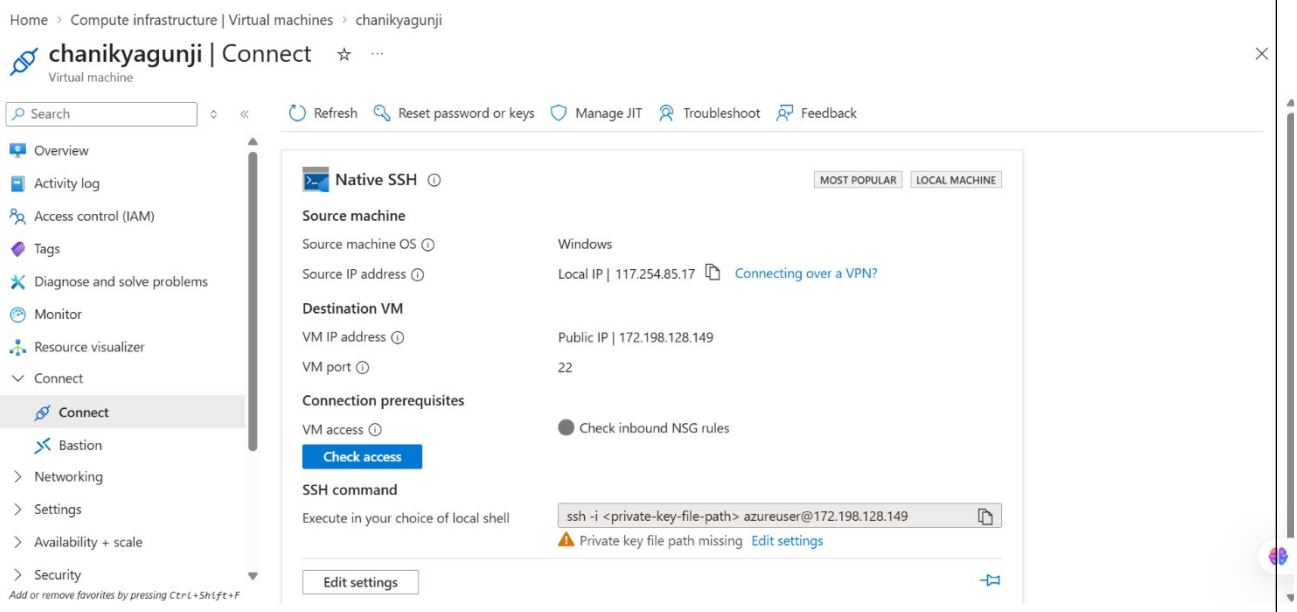
Verify that access is denied when the user lacks the necessary permissions.

Check storage diagnostic logs, if configured, to investigate access attempts.

Question 16: Azure VM Troubleshooting (5 Marks)

A virtual machine is running in Azure, but administrators cannot connect to it. Perform or document the following:

a. Verify the VM's provisioning and power states. (1 mark)



b. Explain how Network Security Group rules can prevent connectivity. (1 mark)

A Network Security Group (NSG) contains inbound and outbound rules that allow or deny network traffic.

For example, an administrator connecting over SSH needs inbound TCP port 22; for Windows Remote Desktop, TCP port 3389 is commonly used.

Connectivity may fail when:

The required port is blocked.

The source IP address is not allowed.

A higher-priority deny rule takes effect.

An outbound rule blocks required return or destination traffic.

c. Explain how subnet and routing configuration affect connectivity. (1 mark)

Subnet: The VM's subnet must have suitable address space and network security rules.

Route table: A user-defined route can send traffic to an incorrect next hop or firewall.

Peering/VPN: Missing routes or incorrect peering settings can prevent access from another network.

Public IP: A VM without a public IP generally cannot be reached directly from the public internet.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet. (2 marks)

Azure Bastion: Connect to the Azure VM securely through the Azure Portal using SSH or RDP without assigning a public IP address to the VM.

Azure Run Command: Run troubleshooting commands directly through the Azure Portal without opening SSH (port 22) or RDP (port 3389) to the public internet.

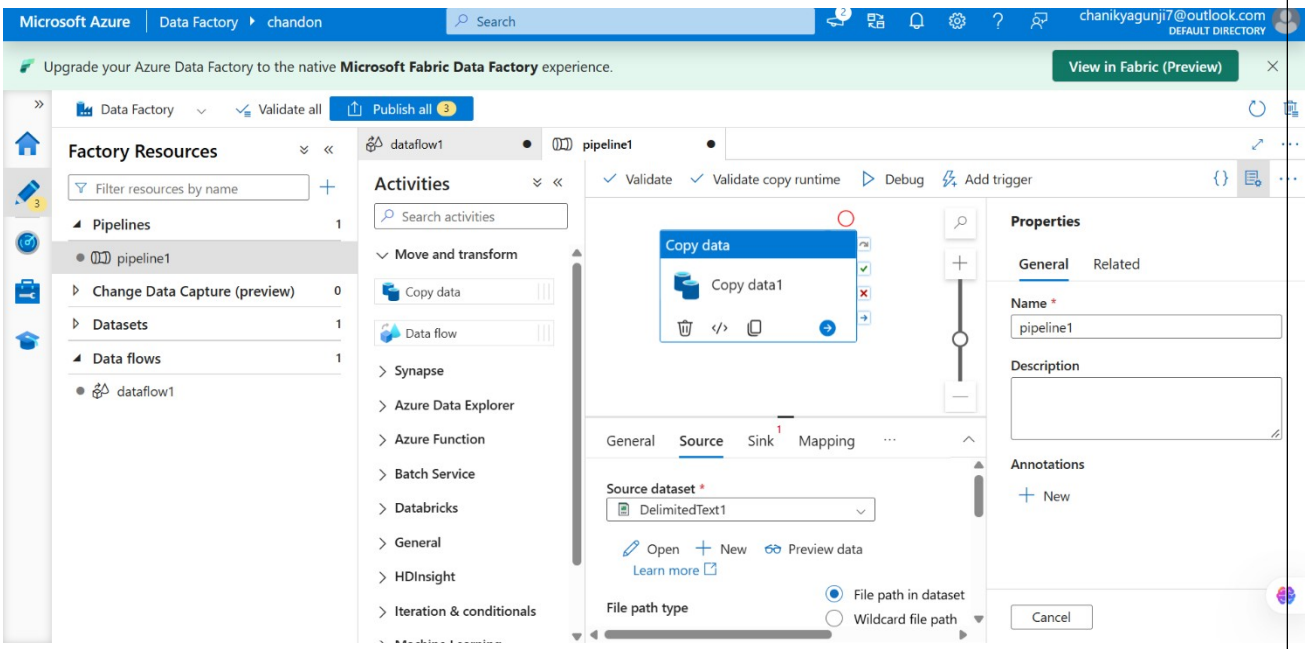
Question 17: Azure Data Integration (5 Marks)

A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement. (1 mark)

Use Azure Data Factory (ADF). It orchestrates data movement and transformation through pipelines, linked services, datasets, and activities.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)



c. Describe how to organize files by ingestion date. (1 mark)

salesdata/

ingestion_date=2026-10-27/

transactions.csv

ingestion_date=2026-10-28/

transactions.csv

In Azure Data Factory, use dynamic content and pipeline date/time expressions to generate the folder path for each run. Include a unique filename or run identifier if multiple files can arrive on the same date.

d. Explain how to handle malformed records and missing values. (1 mark)

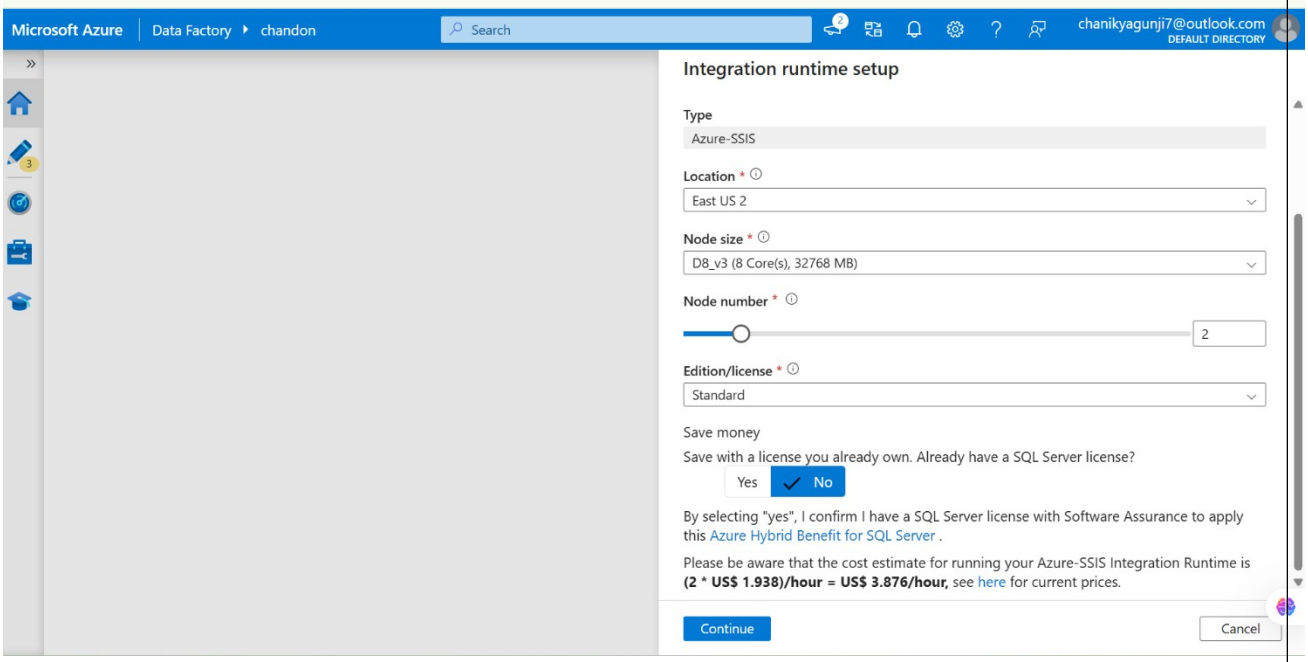
Malformed records: Validate column names, data types, date formats, and CSV structure. Send invalid rows to a separate error or quarantine folder.

Missing values: Replace them with suitable defaults only when appropriate, flag them, or reject records where required fields are missing.

Data transformation: Use Mapping Data Flows, Azure Databricks, or another suitable processing service to clean the data before analysis.

Logging: Record rejected row counts and error reasons

e. Explain how to monitor pipeline execution and validate data completeness. (1 mark)



Open Azure Data Factory Studio and select Monitor.

Check pipeline runs and activity runs for success, failure, and duration.

Inspect error details and rerun failed activities where appropriate.

Compare source and destination row counts.

Validate required columns, null values, duplicate transaction IDs, and total transaction amounts.

Configure alerts through Azure Monitor when needed.